The Alexandria Protocol

Complete Edition · v2.2

Hanns-Steffen Rentschler · 2026

Table of Contents

Abstract
Section I – Self-Binding, Scope, and Epistemic Limits
Section II – Epistemic Premises and Formal Preconditions
Section III – Kantian Meta-Level and Formal Judgment Criteria
Section IV – The Role of Artificial Intelligence: Assessor, Not Authority
Section V – Multi-Model Architecture and Cross-Assessment Cycle
Section V-A – Dual-Builder Architecture [Extension]
- V-A.1 Motivation
- V-A.2 Relationship to Protocol Invariants
- V-A.3 Architectural Overview
- V-A.4 Phase 1: Source Ingestion
- V-A.5 Phase 2: Independent Graph Construction (Builder Alpha & Beta)
- V-A.6 Phase 3: Diff Engine
- V-A.7 Phase 4: Adjudication
- V-A.8 Phase 5: Patch Emission
- V-A.9 Epistemic Status of DBA Output
- V-A.10 What the DBA Does Not Do
- V-A.11 Transition to Section VI

Section VI – Temporal Dimension and Epistemic Persistence
Section VII – Epistemic Uncertainty and Non-Aggregative Statistical Transparency
Section VIII – Blockchain as Epistemic Memory
Section IX – Immutable Persistence of Epistemic Uncertainty
Section X – Epistemic Audit Checklist and Validation Gate
Section XI – Canonical Epistemic State Model
Section XII – Epistemic Interface Layer (Blueprint Studio Reference Implementation)
Section XIII – Result Formats and Epistemic Status
Section XIV – Epistemic Value and Optional Economic Layer Integration
Section XV – Sociological Effects and Conflict Potential

Section XVI – Demarcation, Non-Goals, and Misprojections
Section XVII – Relation to Existing Architectures and Outlook

Appendix A – Threat Model and Adversarial Robustness
Appendix B – End-to-End Epistemic Lifecycle Examples
Appendix C – Formal Guarantees and Structural Invariants
Appendix D – Minimal Reference Implementation Sketch

— Technical Annexes

Technical Annex A – Canonical Graph Schema (v2)
Technical Annex B – Diff Ontology (v2)
Technical Annex C – Adjudication Rulebook (v2)
Technical Annex D – Seal Policy (v2)
Technical Annex E – Claim Format (v2)
Technical Annex F – Relationsontologie
Technical Annex G – Graph Maturity Metrics

G.1–G.7: M1–M5 · Composite Φ · Schwellwerte · Audit Trail · Implementierung

Technical Annex H – v2.2 Structural Changes (Sprint 1–3)

H.1–H.4: Normebenen · Adjudication · BranchNode · Seal · Drei-Ebenen-Audit · Bias-Metadaten · Evaluationsrahmen

Technical Annexes

Implementation Specifications · v2 · Protocol-Compliant

These annexes specify the Dual-Builder Architecture data formats.
They are formally declared extensions. Protocol invariants (Sections I–XVII) remain unchanged.

Technical Annex A – Canonical Graph Schema (v2)
Technical Annex B – Diff Ontology (v2)
Technical Annex C – Adjudication Rulebook (v2)
Technical Annex D – Seal Policy (v2)
Technical Annex E – Claim Format (v2)
Technical Annex F – Relationsontologie

Modern epistemic systems rely on implicit temporal assumptions, opaque validation processes, and authority-based credibility signals. This creates structural vulnerabilities, including unverifiable epistemic lineage, retroactive narrative modification, and difficulty distinguishing stable knowledge from transient or adversarial claims.

This paper introduces the Alexandria Protocol, a formally defined epistemic infrastructure that treats knowledge as a temporally indexed, structurally reconstructible process rather than a static set of conclusions. Alexandria does not attempt to determine truth. Instead, it enforces structural admissibility through explicit derivation paths, temporal anchoring, uncertainty disclosure, and immutable epistemic lineage preservation.

The protocol defines knowledge as a state function over time and constrains all epistemic state transitions through a formally specified Patch Domain-Specific Language (Patch-DSL). Each modification is cryptographically anchored to an append-only chain, enabling complete historical reconstructibility. Validation is performed through a formally defined audit gate that verifies structural integrity, category purity, temporal consistency, and explicit assumption disclosure. Disagreement is handled through mandatory branch formation rather than consensus aggregation or data deletion.

Alexandria introduces formal metrics of epistemic stability and persistence, defined independently of truth probability or institutional authority. Stability reflects validation persistence under adversarial and independent reconstruction, not correctness. The protocol explicitly preserves dissent, invalidation, and epistemic failure as part of permanent epistemic history.

A formal threat model demonstrates that the protocol maintains epistemic integrity under adversarial participation, including malicious submissions, coordinated manipulation attempts, and model-level adversarial behavior. Alexandria guarantees reconstructibility, immutability, and structural transparency, but does not guarantee epistemic correctness or consensus.

The result is a self-binding epistemic protocol that replaces authority-dependent credibility with formally auditable epistemic lineage. Critically, Alexandria replaces the citation pyramid as the primary mechanism of academic credibility: instead of inherited authority through chains of reference, epistemic weight is established through cryptographically verifiable contribution lineage, manipulation-resistant by design. Alexandria provides a generalizable infrastructure for managing evolving knowledge systems in scientific, computational, and distributed artificial intelligence contexts.

Section I – Self-Binding, Scope, and Epistemic Limits

I.1 Starting Point

Alexandria is not a response to any single crisis, not a tool for enforcing particular truths, and not a contribution to existing political, scientific, or moral disputes.

Alexandria arises from a simpler, more fundamental observation:

Modern societies possess ever more knowledge, yet ever less agreement on how knowledge may be assessed at all.

This lack of clarity is currently compensated, inadequately, by:

– Authority (titles, institutions, reputation)
– Morality (attitude, urgency, guilt)
– Power (political, economic, media-based)
– Simulated consensus (majorities, narratives, platform logics)

Alexandria does not address the content of these disputes, but rather their formal preconditions.

I.2 The Scope of Alexandria

Alexandria makes no claim to truth.

Alexandria does not claim:

– to determine what is true
– to separate false from correct positions
– to resolve societal conflicts
– to replace moral judgments
– to legitimize political decisions

The claim is narrower – and at the same time more fundamental:

Alexandria claims to provide a procedure that makes it verifiable under which conditions a judgment about knowledge is admissible at all.

It is not about results, but about assessability.

I.3 Epistemic Self-Binding

Alexandria submits itself to a strict self-limitation.

This self-binding is constitutive and non-negotiable:

Alexandria does not judge truth. It judges exclusively the formal admissibility of epistemic paths.

Alexandria does not replace existing institutions. It provides them with a formal infrastructure.

Alexandria holds no interpretive authority. All results are reconstructive, not authoritative.

Alexandria knows no closure in the sense of final knowledge. Every version remains in principle revisable.

This self-binding is not a weakness, but the core of the architecture.

A system that crosses this boundary would itself become an authority and thereby miss its own purpose.

I.4 Separation of Truth, Judgment, and Power

Alexandria rests on a clear distinction that is widely blurred today:

- Truth is not a system state.
- Judgments are human acts under conditions.
- Power influences which judgments become visible, accepted, or sanctioned.

Alexandria intervenes exclusively at the second level.

It changes not what may be thought, but how judgments about what is thought can be formally examined.

This makes Alexandria critical of power, without itself exercising power.

I.5 Moral Neutrality (Precisely Understood)

Alexandria is morally neutral, but not valueless.

This means:

- Moral arguments are admissible, but only as what they are: normative posits.
- Moral urgency does not replace formal derivation.
- Moral agreement does not produce epistemic validity.

Alexandria does not devalue morality. It merely denies morality the role of an epistemic instrument.

I.6 Consequences of This Self-Positioning

From this self-binding it follows necessarily:

– Alexandria cannot promise to pacify conflicts.
– Alexandria cannot prevent its results from being instrumentalized.
– Alexandria cannot guarantee that its use will be perceived as “fair.”

What Alexandria can achieve is limited – but precise:

It can make visible where judgments rest on cleanly separated assumptions, and where power, morality, or authority obscure this separation.

This making-visible is the sole claim. Everything beyond lies outside the architecture.

I.7 Transition to the Architecture

Only against this background is it meaningful to speak of mechanisms.

The following sections therefore describe:

– not why Alexandria is desirable
– not what results it should produce
– but how an architecture can look that withstands this self-binding.

Section II – Epistemic Premises and Formal Preconditions

II.1 Why Premises Are Necessary

An architecture that aims to make judgments about knowledge assessable cannot be free of presuppositions.

It must disclose which epistemic assumptions it makes – and which it expressly does not make.

Alexandria claims no metaphysical theory of truth.

It does, however, establish formal conditions without which assessability of knowledge is not possible.

These conditions are not a result, but a prerequisite of the protocol.

II.2 First Premise: Truth Is Not a System State

Alexandria assumes:

Truth is not a state that a system can reach or store.

Truth is: not computable, not aggregable, not stably storable.

What systems can achieve is something different:

- Assessability of propositions
- Traceability of derivations
- Transparency of assumptions

Any attempt to operationalize truth itself inevitably leads to authority or power.

II.3 Second Premise: Consensus Is Not a Truth Criterion

Alexandria recognizes consensus as a social phenomenon, not an epistemic one.

Consensus can arise from:

- shared evidence
- shared interests
- moral pressure
- institutional power
- exhaustion

Therefore: Agreement is no proof of validity. Disagreement is no proof of error.

Alexandria treats consensus exclusively as an observable quantity, never as a decision rule.

II.4 Third Premise: Morality Is Not an Epistemic Instrument

Alexandria strictly distinguishes between:

- normative posits (ought-statements)
- descriptive propositions (is-statements)
- model assumptions
- speculative hypotheses

Moral arguments are admissible, but: they produce no epistemic validity.

Morality can generate urgency. It can provide motivation. It can legitimize actions. It cannot, however, replace derivation.

II.5 Fourth Premise: Authority Does Not Replace Assessability

Alexandria acknowledges authority as a social reality, not as an epistemic shortcut.

Titles, institutions, reputation, or historical status:

– can direct attention
– can generate trust
– can coordinate actions

They cannot, however, compensate for a formal error.

A proposition is not more valid because it comes from an authoritative source.

Alexandria treats every source equally – not in the sense of equivalence, but in the sense of equal formal scrutiny.

II.6 Fifth Premise: Knowledge Must Function Under Adverse Conditions

Alexandria does not presuppose good will.

The protocol is explicitly designed to function in the presence of:

– self-interested actors
– strategic behavior
– conflicts of interest
– incomplete information
– deliberate distortion

From this follows:

An epistemic procedure that presupposes moral integrity is not a robust procedure.

Alexandria replaces trust not with distrust, but with structure.

II.7 Sixth Premise: The Path Is Verifiable, the Result Is Not

Alexandria never evaluates propositions in isolation.

The object of assessment is always:

– the epistemic path
– the sequence of assumptions
– the rules of derivation
– the separation of categories

The result is merely the provisional endpoint of a path, not its legitimation.

It is not the result that supports the path. The path supports – or refutes – the result.

II.8 Consequences of These Premises

From these assumptions it follows necessarily:

– There can be multiple consistent results.
– Undecidability is a valid state.
– Dissent is not a malfunction, but an epistemic signal.
– Revision is not failure, but the normal state.

Alexandria thereby replaces the ideal of final truth with the ideal of permanent assessability.

II.9 Transition to the Formal Architecture

The preceding sections have established:

– what Alexandria is not
– which claims it forbids itself
– which epistemic conditions it presupposes

Only now is it possible to describe the architecture itself.

The following section sets out:

– how assessability is formally operationalized
– why no single instance judges
– and why multi-model structures are mandatory

Section III – Kantian Meta-Level and Formal Judgment Criteria

III.1 Why a Meta-Level Is Necessary

A system that seeks to secure the assessability of knowledge must not operate at the level of individual propositions.

It must introduce a level above: a meta-level that determines under which conditions a judgment is admissible at all.

Without this level, the following inevitably arise:

- category confusions
- implicit normativity
- authoritative short-circuits
- moral or political override

Alexandria introduces this meta-level explicitly.

III.2 Transcendental Logic (in the Kantian Sense)

Alexandria follows no substantive theory of truth.

It follows a transcendental logic in the Kantian sense:

Not the truth of a proposition is examined, but the conditions under which it may count as assessable at all.

This shifts the focus: away from the What, toward the How, and above all toward Under which preconditions.

This shift is decisive for keeping authority, morality, and power out of judgment formation.

III.3 Separation of Epistemic Categories

Alexandria demands a strict separation of the following categories:

Empirical propositions: Statements about observable facts, measurements, or data.

Normative propositions: Ought-statements, value judgments, moral or political posits.

Model assumptions: Simplifications, abstractions, or theoretical frameworks.

Speculative hypotheses: Assumptions without secured empirical basis.

This separation is non-negotiable.

Any mixing of these categories counts as a formal error, regardless of the content or moral evaluation of the proposition.

III.4 Formal Judgment Criteria

A judgment in the sense of Alexandria is admissible only when the following criteria are met:

Category purity: Every proposition is unambiguously assigned to an epistemic category.

Explicit assumptions: All presuppositions are disclosed; no implicit premises.

Logical consistency: No internal contradictions in the epistemic path.

Derivation rules: Transitions between propositions are formally justified.

Revisability: Every proposition remains in principle verifiable and correctable.

These criteria concern the path, not the result.

III.5 Judgment vs. Decision

Alexandria strictly distinguishes between:

- Judgments (epistemic)
- Decisions (political, moral, practical)

A judgment describes: which propositions are consistent under given assumptions, where dissent exists, and where undecidability obtains.

A decision makes: a choice, a prioritization, an action.

Alexandria renders judgments. It makes no decisions.

This separation protects the system from political or moral instrumentalization.

III.6 Why Humans Cannot Stably Maintain This Level

In human discourse, this meta-level is regularly violated:

– through rhetorical abbreviation
– through moral urgency
– through institutional pressure
– through time constraints
– through power asymmetries

Alexandria does not replace human judgment, but stabilizes the formal conditions under which it is exercised.

III.7 Transition to the Operative Architecture

The Kantian meta-level establishes:

– what may be examined
– how it may be examined
– what lies explicitly outside the system

Only on this basis can it be described why no single instance may judge, why multiple models are necessary, and how formal consistency can be verified computationally.

The following section therefore describes the role of artificial intelligence in Alexandria – strictly limited and functionally defined.

Section IV – The Role of Artificial Intelligence: Assessor, Not Authority

IV.1 Why Artificial Intelligence Is Necessary

The formal requirements that Alexandria places on assessability – category purity, consistency checks, complete reconstruction of epistemic paths – are not scalable for human actors alone.

Not from lack of intelligence, but because of:

– cognitive limitations
– time pressure
– institutional embeddedness
– social dependencies
– narrative distortion

Alexandria uses artificial intelligence not to replace human thinking, but to maintain formal discipline consistently where humans structurally fail.

IV.2 Principle: No Single AI Judges

Alexandria expressly excludes:

– a deciding instance
– a central model
– an aggregated “AI opinion”

A single AI would merely be a new authority.

Therefore the constitutive principle applies: Judgments arise only from the structured interaction of multiple independent models.

IV.3 Functional Roles of AI Systems

The participating AI systems are not uniform, even though they may be based on similar foundation models. They assume clearly separated roles:

Analysis models: Reconstruct the submitted epistemic path, identify propositions, assumptions, and transitions.

Assessor models: Check formal criteria: category purity, logical consistency, explicit vs. implicit assumptions.

Adversarial models: Deliberately attempt to uncover alternative derivations, counter-assumptions, or hidden contradictions.

Consistency models: Examine whether contradictory assessments can be resolved through assumption changes, precision, or separation.

None of these models decides. They deliver formal judgment contributions.

IV.4 Independence Despite Non-Ideal Model Landscape

Alexandria realistically assumes that genuine diversity of training data, architectures, and target spaces is currently limited.

The protocol responds to this not with denial but with transparency:

Where structural diversity is lacking, functional diversity is enforced.

This means:

– strictly separated roles
– different evaluation rules
– separate prompt and rule sets
– separate iteration paths

This limitation is documented, not concealed.

IV.5 No Truth Judgment, No Consensus Substitute

The output of the AI systems is neither a truth judgment nor a consensus.

It consists of:

– formal audit reports
– identified errors or uncertainties
– explicit assumption lists
– marked dissent

The AI does not answer the question “What is true?” but rather “Under which assumptions is this proposition consistent?”

IV.6 Protection Against AI Authority

Alexandria explicitly recognizes the danger that technical consistency is mistaken for truth.

Therefore the following protective principles apply:

– No unified output
– No numerical overall score
– No ranking of positions
– No automatic recommendation

Every result remains: explainable, reconstructible, fragmented.

Authority arises not through technology, but through opacity. Alexandria relies on radical disclosure.

IV.7 Transition to the Multi-Model Architecture

The role of AI is thereby clearly bounded:

– no authority
– no truth
– no decision

The remaining question is no longer normative, but architectural:

How must multiple AI systems be organized so that formal dissent becomes visible rather than disappearing through aggregation?

The following section describes this organization: the multi-model architecture and the cross-assessment cycle.

Section V – Multi-Model Architecture and Cross-Assessment Cycle

V.1 Why Multi-Model Structures Are Mandatory

A single judgment – even one that appears formally correct – always remains vulnerable to:

– implicit assumptions
– model bias
– blind spots
– hidden normativity

Alexandria therefore proceeds from a simple but rigorous principle:

A judgment is epistemically reliable only if it survives contradiction.

This contradiction must not be external or accidental; it must be structurally enforced.

V.2 Basic Principle of the Multi-Model Architecture

Alexandria organizes judgment formation as a jury process, not as aggregation.

This means: no averaging, no majority vote, no weighting.

Instead:

– parallel, independent assessments
– explicit visualization of deviations
– reconstruction of the reasons for dissent

The goal is not agreement, but explained difference.

V.3 The Claim Graph as Central Unit

The basis of every assessment is a claim graph.

It consists of:

– the proposition to be examined
– all explicit assumptions
– the data or sources used
– the derivation rules between propositions

The claim graph is not a text, but a structured representation of the epistemic path.

Everything that is not part of the graph is not part of the judgment.

V.4 Phase 1: Parallel Assessment

Multiple AI models examine the same claim graph in parallel.

Each model produces:

– a formal audit report
– a list of identified assumptions
– marked category errors
– identified contradictions or gaps

In this phase there is: no exchange, no coordination, no harmonization.

Isolation is not a disadvantage here, but a prerequisite.

V.5 Phase 2: Deviation Detection

The results of the parallel assessments are not merged, but juxtaposed.

Deviations are classified as:

– logical contradiction
– differing assumptions
– conceptual drift
– data or source conflict
– undecidability

This classification is purely formal. It contains no evaluation of which position is “better.”

V.6 Phase 3: Cross-Reconstruction

In the cross-reconstruction phase, each model is compelled to trace the divergent judgments of the others.

A model may maintain a contradiction only if it can demonstrate:

– which assumption diverges
– which category is interpreted differently
– which rule was applied differently

Unexplained dissent is inadmissible.

This phase prevents: rhetorical deflection, semantic evasion, and implicit appeals to authority.

V.7 Phase 4: Reconciliation

Specialized consistency models examine whether the divergent judgments can be transformed into consistent sub-paths through:

– precision
– assumption separation
– decomposition of the proposition

Possible outcomes:

Convergence: A single consistent path remains.

Stable ambiguity: Multiple consistent paths coexist, dependent on assumptions.

Formal undecidability: The available evidence is insufficient.

None of these outcomes counts as failure.

V.8 Phase 5: Termination Criteria

The cross-assessment cycle terminates mandatorily when:

– stable convergence has been reached
– stable ambiguity has been established
– undecidability has been formally justified

Endless iteration is excluded. Standstill here is not failure, but an epistemic state.

V.9 Character of Results

The result of the cycle is not a judgment about truth, but an epistemic status report:

– consistent propositions
– explicit assumptions
– remaining dissent
– identified evidence needs

This report is connectable, but not binding.

V.10 Transition to the Temporal Dimension

The preceding sections have described how epistemic judgments arise: through multi-model assessment, cross-reconstruction, and structured reconciliation. The result is an epistemic status report at a given moment.

But a critical question remains unaddressed: What happens to this status over time?

The current formulation models epistemic evolution through discrete patch operations – structural modifications that transform one knowledge state into the next. This captures change, but it does not model time itself as an independent variable.

Without an explicit temporal dimension, the protocol cannot formally address:

– whether a piece of knowledge persists or decays
– how long a validated result remains reliable

– how epistemic stability is measured over time

– how distributed agents reconcile asynchronous validation

These are not marginal questions. For any system that claims to secure assessability, the temporal behavior of knowledge is as fundamental as its logical structure.

The following section introduces time as a first-class epistemic variable and extends the formal apparatus accordingly.

Placement note

Section V-A is inserted between Section V (Multi-Model Architecture and Cross-Assessment Cycle) and Section VI (Temporal Dimension and Epistemic Persistence).

It presupposes Section V in full and prepares the ground for Section VI by specifying how epistemically compliant graph states are produced before temporal analysis begins.

Section V-A — Dual-Builder Architecture (DBA)

V-A.1 Motivation

Section V establishes that epistemic reliability requires structured contradiction: multiple independent models must assess the same claim graph, and their divergences must be formally reconstructed rather than averaged away.

This principle is sound. It leaves one question unanswered:

How is the claim graph produced in the first place?

In human-driven epistemic processes, graph construction is manual, slow, and subject to the same biases that Section V's multi-model assessment is designed to correct. For large-scale corpora — scientific literature, ontological databases, distributed knowledge sources — manual construction is not feasible.

The Dual-Builder Architecture addresses this gap. It applies the structural logic of Section V — independent parallel processing, typed divergence, formal adjudication — to the graph construction process itself. The result is a protocol-compliant epistemic graph produced without human curation at the construction stage, while preserving all protocol guarantees at the output stage.

V-A.2 Relationship to Protocol Invariants

The DBA is an extension, not a modification. The following protocol invariants remain in force without exception:

– No single instance judges (Section IV.2). Neither Builder Alpha nor Builder Beta is authoritative.

– No aggregation of uncertainty (Section VII.4). Diverging uncertainty values are never averaged.

– Branch formation for unresolvable dissent (Section VII.13.4). The DBA's adjudication layer does not suppress branches; it triggers them.

– Explicit assumptions as protocol requirement (Section VII.13.2). Every sealed claim must carry assumptions[].

– Patch-DSL as the only admissible modification interface (Section VII.13.7). All DBA outputs enter the protocol exclusively as patches.

The DBA introduces three constructs that do not exist in the base protocol: Builder roles, a Diff Engine, and an Adjudication layer. These are declared below.

Declared extensions (not in base protocol)

builder_origin field on Claim nodes (alpha | beta | adjudicated)

Typed Diff as an intermediate epistemic object between Builder output and sealed graph

Adjudication Rulebook (Appendix C) as a formal decision layer prior to Patch-DSL submission

Status values: partially_supported, derived (Appendix E, v2)

V-A.3 Architectural Overview

The DBA operates as a six-phase pipeline. Each phase produces a formally defined output that serves as the input to the next. No phase may be skipped.

Source Ingestion	Builder Alpha	Builder Beta	Diff Engine	Adjudi- cation	Patch Emission
Phase 1	*Phase 2a*	*Phase 2b*	*Phase 3*	*Phase 4*	*Phase 5*

The two Builder phases (2a, 2b) run in strict isolation — no information exchange, no shared intermediate state. This isolation is the structural equivalent of the parallel assessment isolation mandated in Section V.4.

V-A.4 Phase 1: Source Ingestion

The source layer provides the raw epistemic material. The DBA operates on two primary source types:

– Ontological substrate: OpenCyc or equivalent formal ontology, providing concept hierarchies, relation types, and instance definitions.

– Scientific literature corpus: OpenAlex or equivalent bibliographic database, providing works, authors, institutions, and citation networks.

Source ingestion is not an epistemic act. It is a pre-epistemic data preparation step. Sources are not evaluated for correctness at this stage — evaluation is the responsibility of the Builder and Audit phases.

All source references must be preserved as source_refs[] on every node that derives from them. This ensures Source Traceability (Seal Criterion D.2).

V-A.5 Phase 2: Independent Graph Construction (Builder Alpha and Builder Beta)

Two independent AI instances — Builder Alpha and Builder Beta — construct separate knowledge graphs from the same source material. Their independence is structural, not merely procedural.

Builder Alpha	Builder Beta
Operates on identical source corpus	Operates on identical source corpus
No access to Beta's intermediate state	No access to Alpha's intermediate state
No shared prompt during construction	No shared prompt during construction
May use different internal strategies	May use different internal strategies
Produces graph Gα	Produces graph Gβ
Tags all nodes: builder_origin = alpha	Tags all nodes: builder_origin = beta

Each Builder produces a graph that is structurally complete but not yet protocol-sealed. Builder output is epistemically provisional: it has not passed the Audit Gate (Section X) and carries no blockchain anchor.

Builder output must conform to the Canonical Graph Schema (Appendix A, v2): all Claim nodes must carry category, modality, assumptions[], and uncertainty {sigma, ci, n, type}. A Builder that produces claims without explicit assumptions[] produces UNVALIDATED claims by protocol definition.

V-A.6 Phase 3: Diff Engine

The Diff Engine performs a structured comparison of Gα and Gβ. It does not resolve differences — it classifies and records them.

Every divergence between the two graphs is assigned a typed Diff from the Diff Ontology (Appendix B, v2). The complete Diff Ontology covers nine categories:

- B.1 Entity Diffs (missing_entity, alias_mismatch, type_mismatch)
- B.2 Concept Diffs (mapping_mismatch, hierarchy_mismatch, scope_mismatch)
- B.3 Claim Diffs (missing_claim, granularity_mismatch, scope_mismatch, modality_mismatch, quantifier_mismatch, temporal_mismatch)
- B.4 Relation Diffs (relation_mismatch, direction_mismatch, causality_mismatch)

– B.5 Evidence Diffs (source_link_mismatch, evidence_strength_mismatch, citation_scope_mismatch)

– B.6 Uncertainty Diffs (uncertainty_divergence_minor, uncertainty_divergence_major, uncertainty_type_mismatch)

– B.7 Status Diffs (status_mismatch)

– B.8 Assumption Diffs (assumption_mismatch, assumption_scope_mismatch)

– B.9 Category Diffs (category_mismatch)

The Diff Engine produces a complete Diff Report: a typed, ordered list of all divergences. This report is an intermediate epistemic object — it is not yet part of the protocol graph, but it is the input to Phase 4.

The Diff Report has no status. It is neither validated nor sealed. It is a structural description of divergence, not an epistemic judgment about it.

V-A.7 Phase 4: Adjudication

The Adjudication layer applies the Adjudication Rulebook (Appendix C, v2) to the Diff Report. For each typed Diff, a corresponding rule determines the outcome.

Adjudication is not resolution by consensus. It is rule-based structural decision-making. The following outcomes are possible for each Diff:

– Convergence: One Builder's formulation is adopted as the canonical version. Reason is documented in the Judgment Log.

– Refinement: A more precise formulation replaces both Builder versions. Documented as DERIVED_FROM.

– Stable Ambiguity: The Diff cannot be resolved without suppressing legitimate epistemic content. Status = STABLE_AMBIGUITY. Branch is triggered.

– Formal Error: One or both Builder outputs violate a protocol structural rule (e.g. category_mismatch, missing assumptions[]). Status = FORMAL_ERROR. Preserved in Judgment Log.

Every adjudication decision is recorded as a Judgment node in the graph. Judgment nodes carry a blockchain_anchor (Section VIII.7, Seal Criterion D.5). No adjudication decision may be made without a corresponding Judgment node. Adjudication without documentation is inadmissible.

The non-aggregation principle (Section VII.4) applies without exception: where Builder Alpha and Builder Beta produce different uncertainty values (sigma, ci, n), these are never averaged. The larger sigma is preserved as the visible uncertainty value; both original values are preserved in the Judgment record.

V-A.8 Phase 5: Patch Emission

After adjudication, all resolved and stable nodes are emitted as Patch-DSL operations. This is the only point at which DBA output enters the Alexandria protocol. Everything before Phase 5 is DBA-internal.

Each emitted patch must satisfy the canonical Patch structure (Section VII.13.2):

```
PATCH {
  patch_id:          unique identifier
  parent_patch_id:   reference to prior epistemic state
  operation:         ADD | MODIFY | DEPRECATE | BRANCH
  target:            KnowledgeNodeID
  timestamp:         t  (mandatory, monotonic)
  category:          empirical | normative | model | speculative
  assumptions:       explicit assumption references  (mandatory)
  derivation:        formal derivation metadata
  validation:        { validated, decay, refs[] }
  uncertainty:       { sigma, ci, n, type }  (mandatory for probabilistic
claims)
  blockchain_anchor: immutable commit reference
}
```

Patches that do not satisfy all applicable mandatory fields are rejected by the Audit Gate (Section X) and remain UNVALIDATED. The DBA does not bypass the Audit Gate. It produces input for the Audit Gate.

V-A.9 Epistemic Status of DBA Output

DBA output enters the protocol in UNVALIDATED state. It transitions through the standard protocol lifecycle:

– UNVALIDATED → VALIDATED: All five Audit Blocks passed (Section X.3–X.7).

– VALIDATED → SEALED: Temporal stability and structural integrity sufficient (Seal Criteria D.1–D.9).

– Any state → FORMAL_ERROR: Structural violation detected at any stage.

– SEALED → STABLE_AMBIGUITY: Adversarial dissent triggers branch formation.

– Any state → DEPRECATED: Foundational assumption invalidated by new patch.

The DBA does not guarantee that its output will reach SEALED state. It guarantees structural completeness of input to the protocol. The protocol determines admissibility.

V-A.10 What the DBA Does Not Do

The following are explicitly outside the scope of the DBA:

– The DBA does not determine truth. It constructs structurally compliant epistemic graphs.

– The DBA does not replace the Audit Gate. All output passes through Section X.

– The DBA does not aggregate uncertainty. Non-aggregation (Section VII.4) is enforced at Phase 4.

– The DBA does not suppress branches. Unresolvable Diffs become branches, not forced consensus.

– The DBA does not bypass the Patch-DSL. All output is emitted as formally structured patches.

– The DBA does not introduce a new authority. Builder Alpha and Builder Beta are parallel assessors, not arbiters.

These exclusions are not implementation choices. They are protocol-level constraints that the DBA inherits from the base protocol.

V-A.11 Transition to Section VI

Section V-A has described how protocol-compliant epistemic graph states are produced automatically from large-scale source corpora. The DBA pipeline produces a set of UNVALIDATED patches that, after passing the Audit Gate, enter the protocol as formally admissible epistemic states.

What happens to these states over time is the subject of Section VI.

The DBA produces epistemic states at a given moment. Section VI asks: how do these states persist, decay, or evolve as time passes? How is epistemic stability formally measured across the patch chain? How are asynchronous Builder runs reconciled across temporal intervals?

These are not DBA-internal questions. They are protocol-level questions about the temporal behavior of knowledge — which Section VI addresses by introducing time as a first-class epistemic variable.

End of Section V-A — Dual-Builder Architecture

Section VI – Temporal Dimension and Epistemic Persistence

VI.1 Motivation

The Alexandria Protocol as described so far models epistemic evolution through discrete patch operations:

$K_{n+1} = Patch(K_n)$

This formulation captures structural change but does not explicitly model time as an independent variable.

This omission prevents formal treatment of:

– epistemic persistence
– temporal validity
– knowledge decay
– stability over time
– asynchronous validation across distributed agents

To resolve this, we introduce an explicit temporal dimension.

VI.2 Definition: Epistemic Time

Let $T \subseteq \mathbb{R}$ be a totally ordered temporal domain.

Define epistemic state as a time-indexed function:

$$K : T \to S$$

where K(t) is the knowledge state at epistemic time t, and S is the space of valid epistemic configurations.

The tuple E(t) = (K(t), M(t), V(t)) defines the complete epistemic state, where M(t) = metadata and V(t) = validation structure.

VI.3 Temporal Patch Operator

The Patch operator becomes time-dependent:

$$Patch : (E(t), \Delta, t) \to E(t + \Delta t)$$

where Δ is a structural modification and $\Delta t > 0$.

Thus: $E(t + \Delta t) = Patch(E(t), \Delta, t)$

This ensures strict temporal ordering of epistemic evolution.

VI.4 Epistemic Persistence

Define epistemic persistence of a knowledge element k as:

$$Persistence(k, t_0, t_1) = 1 \text{ if } k \in K(t) \text{ for all } t \in [t_0, t_1], \text{ and } 0 \text{ otherwise.}$$

Persistence duration:

$$Duration(k) = \sup \{ t_1 - t_0 \mid Persistence(k, t_0, t_1) = 1 \}$$

This provides a direct measure of epistemic survival.

VI.5 Epistemic Stability

Define stability as the temporal robustness of validated knowledge:

$Stability(k, t) = ValidatedDuration(k, t) / TotalDuration(k, t)$

where $ValidatedDuration(k, t) = \int$ from t_0 to t of $Validation(k, \tau)\, d\tau$

and $Validation(k, \tau) \in \{0, 1\}$.

VI.6 Epistemic Half-Life

Define the epistemic half-life H(k) as:

$H(k) = inf\{ \Delta t \mid P(Validity(k, t + \Delta t)) = 1/2 \}$

This models the expected degradation of epistemic reliability.

Examples:

Physical constants: Very large half-life
Scientific theories: Large half-life
Economic forecasts: Moderate half-life
News claims: Small half-life

VI.7 Temporal Validation Operator

Validation becomes time-dependent:

$VALIDATE : (k, t) \rightarrow \{valid, invalid, uncertain\}$

The Seal operator is extended:

$SEAL(k, t, Stability)$

This allows temporal certification.

VI.8 Epistemic Continuity

Define continuity as integrated stability over time:

$Continuity(k) = \int \text{from } t_0 \text{ to } t_1 \text{ of } Stability(k, t)\, dt$

This provides a scalar measure of long-term epistemic reliability.

VI.9 Branch-Dependent Temporal Evolution

For branches B_i:

$K_i : T_i \rightarrow S$

Branch merge requires temporal reconciliation:

$Merge(K_1(t), K_2(t))$

subject to consistency constraints.

VI.10 Temporal Consistency Condition

Alexandria requires monotonic temporal ordering:

$t_1 < t_2 \Rightarrow State(t_2) \text{ derived from } State(t_1) \text{ under documented transformation rules.}$

No backward epistemic modification is permitted.

This prevents retroactive epistemic corruption.

VI.11 Implementation in Patch-DSL

The Patch structure is extended:

```
PATCH {
    target: KnowledgeNodeID
    operation: ADD | MODIFY | DEPRECATE | BRANCH
    timestamp: t
    stability: optional
    validation: optional
}
```

Timestamp becomes mandatory.

Note: The structure shown here is illustrative. The canonical, normative specification of the Patch-DSL is defined in Section VII.13.

VI.12 Consequences for LLM-Based Systems

This extension enables:

– detection of epistemic drift
– prevention of knowledge degeneration
– temporal ranking of knowledge reliability
– persistent epistemic identity
– cross-agent temporal reconciliation

This resolves a core limitation of stateless language models.

VI.13 Formal Epistemic State Definition

Final form:

$E(t) = (K(t), M(t), V(t), Stability(t), Persistence(t))$

Evolution:

$E(t + \Delta t) = Patch(E(t), \Delta, t)$

VI.14 Summary and Transition

The introduction of explicit epistemic time transforms Alexandria from a discrete patch system into a temporally coherent epistemic process.

This enables formal treatment of: knowledge persistence, stability, temporal validity, and epistemic continuity. Time becomes a first-class epistemic variable.

However, temporal persistence alone does not fully characterize epistemic reliability. Some propositions are not only time-dependent but uncertainty-dependent. This uncertainty does not arise from logical inconsistency, but from probabilistic structure, incomplete observation, or statistical estimation.

The following section extends the formal apparatus to make such uncertainty explicitly visible – without interpreting it as truth probability, and without permitting its concealment through aggregation.

Section VII – Epistemic Uncertainty and Non-Aggregative Statistical Transparency

VII.1 Motivation

The temporal extension introduced in Section VI establishes epistemic time as a first-class variable. However, temporal persistence alone does not fully characterize epistemic reliability.

Some propositions are not only time-dependent but uncertainty-dependent.

This uncertainty does not arise from logical inconsistency, but from probabilistic structure, incomplete observation, or statistical estimation.

Alexandria does not resolve such uncertainty. It makes it visible.

To preserve its epistemic self-binding, Alexandria does not interpret uncertainty as truth probability. Instead, uncertainty is treated strictly as metadata of epistemic assessability.

VII.2 Separation of Structural and Statistical Epistemic Properties

Alexandria distinguishes strictly between:

Structural epistemic properties:

– logical consistency
– category purity
– derivational traceability
– temporal persistence

Statistical epistemic properties:

– probabilistic variance
– confidence intervals
– sample size dependence
– estimation uncertainty

Structural properties determine assessability. Statistical properties characterize uncertainty within assessable probabilistic propositions.

Statistical uncertainty never replaces structural validation.

VII.3 Definition: Epistemic Uncertainty Metadata

For each knowledge element k, define epistemic uncertainty metadata:

$U(k) = (\sigma, CI, n)$ *if probabilistic;* $\varnothing$ *if deterministic*

where σ = standard deviation, CI = confidence interval, n = sample size.

This metadata describes uncertainty without altering epistemic validity.

A proposition may be:

- structurally valid and statistically uncertain
- structurally valid and statistically precise
- structurally invalid regardless of statistical certainty

Statistical precision does not repair structural errors.

VII.4 Non-Aggregative Uncertainty Principle

Alexandria explicitly forbids aggregation of uncertainty across models into a single averaged certainty value.

Instead, uncertainty is preserved in its distributed form.

Define system uncertainty as:

$U_system = \{ U_1(k), U_2(k), ..., U_n(k) \}$

not:

$\bar{U}(k)$

This prevents artificial certainty generated by averaging disagreement.

Uncertainty is preserved, not smoothed.

VII.5 Worst-Case Uncertainty Visibility Rule

When uncertainty differs across models, the system preserves the maximum uncertainty as part of the epistemic status report:

$\sigma_visible(k) = max(\sigma_1, \sigma_2, ..., \sigma_n)$

This does not invalidate lower-uncertainty estimates. It prevents concealment of uncertainty.

This rule ensures epistemic transparency under adversarial or incomplete conditions.

VII.6 Uncertainty Propagation Constraint

Uncertainty must propagate through epistemic derivations.

If a proposition k_2 depends on uncertain input k_1, then:

$U(k_2) \geq f(U(k_1))$

where f is a derivation-dependent uncertainty propagation function.

This prevents epistemic certainty from emerging without justification.

Certainty cannot arise from uncertain premises without explicit justification.

VII.7 Compatibility with Category Purity

Statistical uncertainty applies only to probabilistic propositions.

It does not apply to:

- normative propositions
- logical validity
- category assignments
- structural derivation rules

Attempting to assign statistical certainty to normative or structural propositions constitutes a category violation.

Alexandria detects and marks such violations.

VII.8 Integration with Temporal Stability

Uncertainty and temporal persistence are orthogonal dimensions.

Define epistemic reliability as a two-dimensional function:

$R(k, t) = (Stability(k, t), U(k, t))$

This allows distinction between:

– stable but uncertain knowledge
– precise but unstable knowledge
– stable and precise knowledge
– unstable and uncertain knowledge

Alexandria does not collapse these dimensions. Both remain visible.

VII.9 Implementation in Patch-DSL

Uncertainty metadata is included when applicable:

```
PATCH {
    target: KnowledgeNodeID
    operation: ADD | MODIFY | DEPRECATE | BRANCH
    timestamp: t
    uncertainty: {
      type: probabilistic | deterministic
      sigma: optional
      confidence_interval: optional
      sample_size: optional
    }
}
```

Uncertainty metadata is mandatory only for probabilistic propositions. It is forbidden for deterministic structural propositions.

Note: The structure shown here is illustrative. The canonical, normative Patch-DSL specification, including all mandatory and optional fields, is defined in Section VII.13.

VII.10 Consequences for Epistemic Integrity

This extension ensures:

– uncertainty cannot be concealed through aggregation
– statistical precision cannot override structural errors
– disagreement remains visible

– probabilistic knowledge remains reconstructible

– epistemic transparency extends to probabilistic domains

Alexandria thereby preserves its self-binding while extending assessability to probabilistic knowledge.

VII.11 Relation to Epistemic Self-Binding

This section does not introduce probabilistic truth evaluation. It introduces probabilistic transparency.

Alexandria does not determine probability of truth. It determines assessability and visibility of uncertainty.

This distinction preserves the protocol's fundamental self-limitation.

VII.12 Transition to Patch-DSL

The preceding subsections defined the structural, temporal, and statistical properties of epistemic uncertainty. However, these definitions alone do not specify how epistemic states are operationally modified.

For Alexandria to function as a distributed epistemic system, all state modifications must occur through a formally constrained, reconstructible interface. The following subsection defines this interface.

VII.13 Patch-DSL as Canonical Epistemic Modification Interface

The Patch-DSL (Domain-Specific Language) defines the only admissible mechanism by which epistemic state transitions may occur.

No epistemic modification exists outside this formal structure.

VII.13.1 Atomicity of Epistemic Modification

Every epistemic modification is represented as an atomic patch:

$E(t^{+}) = \Delta E(t), \boxtimes, t)$

Each patch represents: a structural modification, a temporal event, and a formally auditable object.

Patches are immutable once committed.

VII.13.2 Canonical Patch Structure

Each patch SHALL include:

```
PATCH {
    patch_id: unique identifier
    parent_patch_id: reference to prior epistemic state
    operation: ADD | MODIFY | DEPRECATE | BRANCH
    target: KnowledgeNodeID
    timestamp: t
    category: empirical | normative | model | speculative
    assumptions: explicit assumption references
    derivation: formal derivation metadata
    validation: optional validation metadata
    uncertainty: optional statistical uncertainty metadata
    blockchain_anchor: immutable commit reference
}
```

Fields SHALL NOT be omitted where applicable. Implicit assumptions constitute a formal protocol violation.

VII.13.3 Temporal Binding Requirement

Every patch MUST include a timestamp. This ensures: strict temporal ordering, prevention of retroactive epistemic modification, and reconstructible epistemic evolution.

No patch may modify prior epistemic states retroactively.

VII.13.4 Branch Formation Requirement

When structural dissent cannot be resolved through assumption reconciliation, a branch MUST be created.

Branch formation ensures: dissent preservation, assumption separation, and prevention of epistemic collapse into forced consensus.

Branching is not an exception. It is a normal epistemic operation.

VII.13.5 Uncertainty Visibility Requirement

When probabilistic propositions are modified, uncertainty metadata SHALL be included:

$U(k) = (\sigma, CI, n)$

This ensures statistical transparency. Uncertainty metadata SHALL NOT be aggregated across patches.

VII.13.6 Validation Metadata

Validation metadata MAY include: validation function reference, stability index, and cross-assessment references.

Validation metadata describes validation structure. It does not determine truth.

VII.13.7 Epistemic Integrity Condition

The epistemic integrity of Alexandria requires: all epistemic state transitions MUST be reconstructible exclusively through Patch-DSL operations.

Any epistemic modification not represented as a patch SHALL be treated as non-existent within the protocol.

VII.13.8 Consequence

The Patch-DSL transforms Alexandria from a descriptive epistemic architecture into an operational protocol.

Knowledge becomes: structurally explicit, temporally anchored, statistically transparent, and historically reconstructible.

Epistemic evolution becomes a formally auditable process.

VII.14 Transition to Blockchain Anchoring

With epistemic time (Section VI), epistemic uncertainty (Section VII.1–12), and the canonical modification interface (Section VII.13) now formally defined, a critical architectural question arises: how are these epistemic states – including their temporal evolution, their uncertainty metadata, and the patch chain itself – preserved so that they cannot be retroactively altered, selectively quoted, or narratively distorted?

The following section introduces the blockchain as the immutable memory of Alexandria's epistemic processes.

Section VIII – Blockchain as Epistemic Memory

VIII.1 Why an Immutable Memory Is Necessary

Knowledge fails in modern societies rarely due to lack of analysis, but due to retroactive reinterpretation.

Typical distortions include:

- retrospective smoothing ("That was always clear")
- selective citation
- moral or political rewriting
- retroactive authority construction

A system that aims to secure assessability must therefore not only judge, but historicize knowledge.

VIII.2 Function of the Blockchain (Clearly Bounded)

The blockchain in Alexandria does not serve:

- truth anchoring
- decentralization for its own sake
- automation of decisions
- legitimation of results

Its sole function is: immutable reconstructibility of epistemic paths over time.

VIII.3 What Is Stored

Only formal metadata is stored, not content in the narrow sense:

– versions of claim graphs
– branching points in the epistemic path
– explicit assumptions
– points of dissent
– termination criteria and reasons
– timestamps and sequences

The blockchain does not store what is “correct,” but how something came about.

VIII.4 What Is Expressly Not Stored

Not stored are:

– moral evaluations
– rankings of results
– majority ratios
– political or institutional attributions

This keeps the memory epistemically neutral.

VIII.5 Effect on Power and Authority

Through this form of storage, the following become impossible:

– retroactive heroization
– reconstruction of genius
– moral appropriation
– institutional historiography

No one can claim they “were always right” when the path is publicly reconstructible.

VIII.6 Time as a Component of Knowledge

Alexandria treats time not as a disturbance, but as an integral component of knowledge.

A proposition is assessed not only by its formal consistency, but also by:

– the point in time
– the available evidence
– the assumptions prevailing at the time

This makes knowledge historically honest.

VIII.7 Patch Chain Anchoring and Epistemic Chain Integrity

The preceding subsections defined blockchain anchoring as the persistence substrate of epistemic states. However, epistemic persistence alone does not guarantee epistemic chain integrity.

To ensure complete reconstructibility, every epistemic modification MUST be cryptographically anchored as part of a continuous patch chain. This creates an immutable epistemic lineage.

VIII.7.1 Patch Chain Definition

Let P_i denote the i-th epistemic patch. The epistemic chain SHALL be defined as:

$C_i = Hash(P_i, C_{i-1})$

where C_0 is the genesis epistemic state.

Each patch SHALL include a reference to the immediately preceding chain state. This creates a continuous cryptographic chain.

VIII.7.2 Chain Integrity Requirement

Every patch MUST include:

```
blockchain {
    commit_hash: Hash(PATCH, previous_commit_hash)
    previous_commit_hash: reference to prior commit
}
```

The commit hash SHALL uniquely identify the epistemic state transition. Modification of any prior patch SHALL invalidate the chain.

Thus, retroactive epistemic modification becomes cryptographically detectable.

VIII.7.3 Audit Anchoring Requirement

Audit results SHALL be anchored as independent blockchain entries. This ensures: audit integrity, audit traceability, and audit non-repudiation.

Audit records SHALL reference the patch they evaluate. Patches SHALL reference audit records where validation metadata exists. This creates a bidirectional integrity relationship.

VIII.7.4 Branch Anchoring Requirement

When branch formation occurs, each branch SHALL produce a new chain segment. Branch chains SHALL reference their parent chain state.

This preserves complete epistemic lineage across divergent paths. Branching SHALL NOT break reconstructibility. Branching SHALL extend chain structure.

VIII.7.5 Epistemic State Reconstruction Requirement

Given the patch chain $C_0, C_1, C_2, \ldots, C_n$, the epistemic state at time t_n SHALL be reconstructible by sequential application of all patches:

$$E(t_n) = \mathscr{P}_n(\mathscr{P}_{n-1}(\ldots\mathscr{P}_1(E(t_0))\ldots))$$

Blockchain anchoring guarantees that this reconstruction cannot be altered.

VIII.7.6 Tamper Detection Guarantee

Any modification to patch content, validation metadata, audit metadata, or uncertainty metadata SHALL result in chain hash mismatch.

This provides cryptographic detection of epistemic tampering. The protocol guarantees tamper detectability. It does not require trust in any individual participant.

Trust is replaced by verifiable structural integrity.

VIII.7.7 Epistemic Persistence Guarantee

Blockchain anchoring ensures: epistemic history is immutable.

No epistemic state transition may be erased. Invalidated states remain visible as deprecated states. This preserves complete epistemic lineage.

VIII.7.8 Independence from Blockchain Implementation

The Alexandria Protocol does not require a specific blockchain implementation. Any persistence substrate SHALL be admissible if it satisfies:

– immutability

– cryptographic verifiability

– append-only structure

– public or auditable accessibility

Implementation flexibility preserves protocol portability.

Admissible substrate classes include, but are not limited to:

– public blockchains (e.g. Ethereum, Bitcoin)

– permissioned blockchain systems (e.g. Hyperledger)

– cryptographically verifiable append-only logs (e.g. Certificate Transparency logs, Merkle DAG structures)

The choice of substrate is an implementation decision, not a protocol constraint.

VIII.8 Transition to Immutable Uncertainty Persistence

The blockchain now anchors not only epistemic states but the complete patch chain, including audit records. However, Section VII introduced epistemic uncertainty as formal metadata. Uncertainty that is not immutably preserved remains vulnerable to retrospective reinterpretation.

A proposition whose uncertainty was high at time t_0 may later appear artificially certain if its earlier uncertainty is not permanently recorded. This creates a structural vulnerability: not falsification of knowledge, but falsification of the history of knowledge.

The following section addresses this by treating uncertainty itself as an immutable epistemic object.

Section IX – Immutable Persistence of Epistemic Uncertainty

IX.1 Motivation

Section VI introduced epistemic time. Section VII introduced epistemic uncertainty as metadata. However, uncertainty that is not immutably preserved remains vulnerable to retrospective reinterpretation.

A proposition whose uncertainty was high at time t_0 may later appear artificially certain if its earlier uncertainty is not permanently recorded.

This creates a structural vulnerability: not falsification of knowledge, but falsification of the history of knowledge.

Alexandria prevents this by treating uncertainty itself as an immutable epistemic object.

IX.2 Uncertainty as a First-Class Historical Object

For each epistemic state:

$E(t) = (K(t), M(t), V(t), U(t))$

uncertainty metadata U(t) is preserved alongside structural knowledge.

Define the historical uncertainty trace:

$U_H(k) = \{ U(k, t_0), U(k, t_1), U(k, t_2), \dots \}$

This forms an immutable uncertainty trajectory. This trajectory cannot be retroactively modified.

IX.3 Immutability Requirement

For any knowledge element k:

$U(k, t_n)$, once sealed, satisfies: $\forall\, t_m > t_n : U(k, t_n)$ remains accessible and unaltered.

Uncertainty may decrease in later states. It cannot disappear from the historical record.

This prevents retrospective certainty projection.

IX.4 Blockchain Anchoring of Uncertainty Metadata

Each epistemic state commit includes:

$Commit = Hash(K(t), M(t), V(t), U(t), timestamp)$

The blockchain stores:

- structural claim graph state

– validation metadata
– uncertainty metadata
– timestamp

This ensures that both knowledge and its uncertainty remain reconstructible.

The blockchain anchors epistemic uncertainty, not epistemic truth.

IX.5 Prevention of Retrospective Epistemic Smoothing

Without immutable uncertainty storage, the following distortion becomes possible:

Observed sequence: $\sigma(k, t_0)$ = high → $\sigma(k, t_1)$ = medium → $\sigma(k, t_2)$ = low

Retrospective narrative distortion: “Knowledge was always certain.”

Blockchain persistence prevents this. The actual trajectory remains visible:

$U_H(k) = (high \rightarrow medium \rightarrow low)$

This preserves epistemic honesty.

IX.6 Branch-Dependent Uncertainty Preservation

When epistemic branches diverge (B_1, B_2, … B_n), each branch preserves its own uncertainty trajectory:

$U_H^{B_i}(k)$

Branch merge does not overwrite uncertainty. It produces a new uncertainty state U(k, t_merge) while preserving all prior branch uncertainty histories.

IX.7 Epistemic Integrity Condition

Define the epistemic integrity constraint:

Integrity(k) = True if and only if $\forall\, t : U(k, t)$ *is historically reconstructible.*

Loss of uncertainty history constitutes an epistemic integrity violation.

Alexandria prevents such violations by design.

IX.8 Compatibility with Epistemic Self-Binding

The blockchain does not validate knowledge. It preserves epistemic history.

This includes: structural derivations, dissent, validation states, and uncertainty states.

The blockchain strengthens reconstructibility, not authority.

Alexandria remains a system of assessability, not truth certification.

IX.9 Consequences for Epistemic Systems

This extension prevents:

- retrospective certainty projection
- institutional rewriting of uncertainty
- concealment of past epistemic instability
- narrative falsification of epistemic development

It ensures that knowledge remains historically honest.

IX.10 Unified Epistemic Persistence Model

The final epistemic state definition becomes:

$E(t) = (K(t), M(t), V(t), U(t), Stability(t), Persistence(t))$

Stored immutably as:

$BlockchainCommit(E(t))$

This completes the temporal and probabilistic extension of Alexandria.

IX.11 Transition to the Epistemic Audit Gate

With the structural apparatus (Sections I–V), the temporal dimension (Section VI), the uncertainty framework and Patch-DSL (Section VII), the immutable memory and patch chain anchoring (Section VIII), and the persistence of uncertainty (Section IX) now in place, the architectural foundation is structurally complete.

However, one critical mechanism remains undefined: the formal gate through which epistemic patches transition from unvalidated to validated or sealed states. Without this gate, the protocol describes structure but not admissibility.

The following section defines this validation gate.

Section X – Epistemic Audit Checklist and Validation Gate

X.1 Purpose

The Epistemic Status Report describes the structural state of epistemic elements. However, the report itself does not determine admissibility.

Admissibility requires formal audit.

The Epistemic Audit Checklist defines the validation gate through which epistemic patches must pass before transitioning to validated or sealed states.

This checklist ensures that epistemic state transitions remain structurally admissible, temporally anchored, and reconstructible.

The checklist evaluates formal structure, not correctness of conclusions.

X.2 Audit Requirement

Every patch MUST undergo audit evaluation prior to entering VALIDATED or SEALED states.

If audit evaluation is incomplete, the patch SHALL remain in the UNVALIDATED state.

Audit failure SHALL NOT erase or modify the patch. Audit failure SHALL result in a structurally preserved audit failure record.

X.3 Audit Block I – Category Purity Verification

The audit SHALL verify that every epistemic proposition possesses a valid and unambiguous category assignment.

Valid categories include: empirical, normative, model, speculative.

Category assignment SHALL NOT be ambiguous. Category mixing SHALL be treated as a structural violation.

Normative claims SHALL NOT be presented as empirical observations. Model assumptions SHALL NOT be presented as empirical measurements.

Violations SHALL result in audit failure.

X.4 Audit Block II – Path Reconstruction Verification

The audit SHALL verify that the epistemic derivation path is fully reconstructible. This requires:

- explicit assumption references
- explicit derivation metadata
- valid content hash reference

Implicit assumptions SHALL constitute a formal audit failure.

Authority references SHALL NOT substitute for reconstructible derivation. Institutional origin SHALL NOT influence admissibility.

X.5 Audit Block III – Temporal Integrity Verification

The audit SHALL verify temporal admissibility. This requires:

- valid timestamp
- valid parent patch reference
- monotonic temporal ordering

Retroactive epistemic modification SHALL be prohibited. The epistemic state SHALL evolve exclusively forward in time.

X.6 Audit Block IV – Cross-Assessment Verification

The audit SHALL verify that cross-assessment references exist where required.

Cross-assessment MAY include: independent validation models, adversarial models, and replication references.

Dissent SHALL NOT invalidate patches. Dissent SHALL trigger branch formation when structural reconciliation is not possible.

X.7 Audit Block V – Uncertainty Disclosure Verification

Where probabilistic claims exist, uncertainty metadata SHALL be present. This SHALL include: σ, confidence interval, and sample size.

Absence of required uncertainty metadata SHALL prevent transition to validated or sealed states.

Deterministic claims SHALL NOT include probabilistic uncertainty metadata.

X.8 Audit Result States

Audit evaluation SHALL produce one of the following structural states:

UNVALIDATED: Audit incomplete or pending.

VALIDATED: Audit complete, structural admissibility satisfied.

SEALED: Audit complete, temporal stability and structural integrity sufficient for transfer-level admissibility.

FORMAL_ERROR: Structural violation detected.

Formal error SHALL NOT erase epistemic structure.

Admissible state transitions:

UNVALIDATED → VALIDATED (audit passes all structural requirements)

UNVALIDATED → FORMAL_ERROR (structural violation detected)

VALIDATED → SEALED (temporal stability and structural integrity sufficient)

SEALED → STABLE_AMBIGUITY (adversarial dissent triggers branch formation)

SEALED → DEPRECATED (foundational assumption invalidated)

STABLE_AMBIGUITY → DEPRECATED (global refutation across all branches)

No other transitions are admissible. Reverse transitions (e.g. SEALED → UNVALIDATED) are prohibited.

X.9 Audit Independence Requirement

Audit evaluation SHALL depend exclusively on structural criteria.

Audit evaluation SHALL NOT depend on: institutional origin, speaker identity, rhetorical force, or consensus pressure.

Epistemic admissibility is determined solely by structural reconstructibility.

X.10 Audit Preservation Requirement

Audit results SHALL be preserved as part of epistemic history. Audit records SHALL be immutable once committed.

This ensures full reconstructibility of epistemic validation history.

X.11 Transition to the Canonical State Model

With the audit gate now formally defined, the complete operative pipeline is in place: epistemic modification (Patch-DSL), formal validation (Audit Checklist), immutable storage (Blockchain), and uncertainty persistence.

Before turning to output formats and societal effects, it is necessary to consolidate what has been constructed into a single, canonical formal representation. This is not an expansion of claims, but a rendering explicit of what already exists.

Section XI – Canonical Epistemic State Model

XI.1 Purpose

The preceding sections have introduced: structural assessability, multi-model cross-assessment, epistemic time, statistical transparency, and immutable historical storage.

What follows is not an expansion of claims, but a consolidation.

This section provides a canonical formal representation of the Alexandria epistemic state.

It introduces no new authority. It merely renders explicit what has already been constructed.

XI.2 The Epistemic State

For each epistemic time $t \in T$, define the Alexandria state:

$E(t) = (G(t), A(t), C(t), V(t), U(t), S(t), P(t), B(t))$

where:

G(t) – Claim Graph: Structured representation of propositions and derivational relations.

A(t) – Assumption Set: Explicitly declared premises, fully referenceable.

C(t) – Category Mapping: Assignment of each proposition to one of the epistemic categories: empirical, normative, model-based, or speculative.

V(t) – Validation Structure: Multi-model assessment reports and formal checks.

U(t) – Uncertainty Metadata: Statistical descriptors where applicable.

S(t) – Stability Field: Time-weighted validation persistence.

P(t) – Persistence Field: Temporal survival intervals of knowledge elements.

B(t) – Immutable Anchoring: Blockchain-based historical commit of the entire state.

This tuple defines the complete epistemic configuration at time t.

It is descriptive, not normative.

Epistemic identity:

Each knowledge element k is uniquely identified across time and branches by:

$ID(k) = Hash(initial_patch, lineage)$

where initial_patch is the genesis patch that introduced k, and lineage is the ordered sequence of patch references leading to the current state. This ensures that a knowledge element remains uniquely identifiable even when modified, branched, or deprecated.

XI.3 Dynamic Evolution

Alexandria evolves through formally defined patches:

$E(t^{+}) = (E(t), \Delta, t)$

where Δ is an atomic structural modification.

No state transition is implicit. No modification is retroactive. Every change is time-indexed.

Time is not decoration. It is ordering.

XI.4 Stability as Temporally Weighted Validation

For each knowledge element k, define the validation function $v_k(t) \in [0, 1]$ as:

$v_k(t) = (\text{number of validating models at } t) / (\text{total evaluating models at } t)$

where a validating model is one whose independent structural assessment confirms admissibility of k at time t. This is a ratio of structural confirmations, not a truth probability.

Stability is defined as:

$S_k(t) = \int \text{from } t_0 \text{ to } t \text{ of } v_k(\tau) \cdot e^{(-\lambda_k(t-\tau))}\, d\tau \,/\, \int \text{from } t_0 \text{ to } t \text{ of } e^{(-\lambda_k(t-\tau))}\, d\tau$

This formulation ensures:

– recent validation weighs more than distant validation
– epistemic trust decays continuously
– stability is bounded between 0 and 1

The decay constant λ_k defines epistemic half-life:

$H_k = \ln(2) / \lambda_k$

No proposition is timeless by default.

Decay constant selection:

The decay constant λ_k is not globally fixed. It MAY depend on:

– epistemic domain (e.g. physical constants vs. economic forecasts)
– uncertainty level of the knowledge element
– validation frequency (elements validated more frequently decay more slowly)

The selection of λ_k is a domain-specific implementation decision, not a protocol-level constant. The protocol requires only that $\lambda_k > 0$ and that its value is explicitly declared as part of the epistemic metadata.

Critical clarification:

Stability is a structural persistence metric. It does not represent truth probability, correctness likelihood, or predictive accuracy. Stability reflects only validation survival under adversarial and independent structural verification. A stability value of 0.9 means that the proposition has consistently passed structural audits over time – not that it is 90% likely to be true.

XI.5 Persistence

Persistence measures uninterrupted presence of an element:

$P_k(t_0, t_1) = 1$ *if and only if* $k \in K(\tau)$ *for all* $\tau \in [t_0, t_1]$

Duration of survival is recorded, not interpreted.

Persistence does not imply correctness. It implies continuity.

XI.6 Statistical Transparency

For probabilistic propositions only:

$U_k(t) = (\sigma_k(t), CI_k(t), n_k(t))$

For deterministic propositions:

$U_k(t) = \varnothing$

Statistical uncertainty: never replaces structural validation, never overrides category purity, is never aggregated into a single certainty value.

When multiple models produce uncertainty values:

$\sigma_k^{vtstbe}(t) = max_i\, \sigma_{k,i}(t)$

This rule preserves epistemic caution without imposing judgment.

XI.7 Non-Aggregation Principle

Validation reports remain distributed:

$V(t) = \{ V_1(t), V_2(t), \ldots, V_n(t) \}$

There is no averaged opinion. There is no unified verdict.

Convergence may occur. It is never manufactured.

XI.8 Status Classification

The epistemic status of an element is derived from structural conditions.

Possible states include:

- Formal Error

– Undecidable
– Stable Ambiguity
– Structurally Sealed

These states describe: consistency, category purity, persistence, stability, and uncertainty visibility.

They do not declare truth.

XI.9 Immutable Historical Anchoring

Each state produces a commit:

$B(t) = Hash(G(t), A(t), C(t), V(t), U(t), S(t), P(t), t)$

This ensures:

– historical reconstructibility
– preservation of uncertainty
– preservation of dissent
– impossibility of retrospective smoothing

The blockchain anchors process, not result.

XI.10 Final Form

The Alexandria epistemic system may therefore be summarized as:

$E(t) = (G, A, C, V, U, S, P, B)$

with

$E(t^{+}) = (\mathcal{E}(t), \Delta, t)$

and stability defined through exponentially weighted validation persistence.

This formulation: does not compute truth, does not aggregate consensus, does not authorize decisions.

It renders epistemic structure visible over time.

Nothing more.

Scope of guarantee:

Alexandria guarantees reconstructibility and formal admissibility. It does not guarantee correctness, accuracy, or predictive reliability. The protocol ensures that epistemic paths are structurally transparent and historically immutable – not that their conclusions are true. This distinction is fundamental and non-negotiable.

XI.11 Transition to the Epistemic Interface Layer

The canonical state model provides the formal object that Alexandria operates on. However, effective human interaction with this object requires an interface capable of representing epistemic structure explicitly.

The following section defines the requirements for such an interface, using the Blueprint Studio as a reference implementation.

Section XII – Epistemic Interface Layer (Blueprint Studio Reference Implementation)

XII.1 Definition

The Alexandria Protocol defines epistemic state structure and admissibility criteria. It does not prescribe a specific human interaction modality.

However, effective human interaction requires an interface capable of representing epistemic structure explicitly.

The Blueprint Studio is defined as a reference implementation of an Epistemic Interface.

An Epistemic Interface is a system that allows human actors to construct, inspect, and modify epistemic graphs in compliance with Alexandria structural constraints.

The interface does not alter protocol rules. It exposes protocol structure.

XII.2 Functional Mapping Between Protocol and Interface

The Epistemic Interface SHALL provide functional correspondence to Alexandria protocol primitives:

Claim Graph → Visual node graph (“Epistemic Arena”)

Category Assignment → Typed structural modules

Uncertainty Metadata (σ, CI) → Visual uncertainty indicators

Patch Chain → Immutable revision history log

Branch Formation → Explicit structural graph branching

Audit Gate → Validation feedback layer

The interface SHALL NOT conceal epistemic uncertainty. The interface SHALL preserve explicit assumption visibility.

XII.3 Compiler Function

The Epistemic Interface acts as a structural compiler. Human-generated epistemic structures SHALL be compiled into Patch-DSL representations.

Compilation SHALL verify:

– category completeness

– assumption declaration

– structural consistency

– temporal admissibility

Compilation failure SHALL prevent patch emission. Compilation success produces a valid protocol patch.

The interface SHALL NOT bypass protocol validation requirements.

XII.4 Protocol–Interface Separation Invariant

The Epistemic Interface SHALL be logically separable from the Alexandria Protocol.

The protocol operates independently of any specific interface implementation. Blueprint Studio is a reference implementation, not a protocol dependency.

Alternative interfaces MAY exist. Protocol integrity SHALL NOT depend on a specific interface.

XII.5 Interface Transparency Requirement

The Epistemic Interface SHALL preserve full epistemic transparency. This includes:

– explicit uncertainty representation

– explicit dissent visibility

– explicit epistemic lineage visibility

The interface SHALL NOT collapse epistemic branches into single conclusions. The interface SHALL preserve epistemic multiplicity.

XII.6 Interface Role in Epistemic Workflow

The Epistemic Interface facilitates: epistemic graph construction, patch generation, epistemic audit interaction, and epistemic state inspection.

The interface does not determine epistemic admissibility. Admissibility is determined exclusively by protocol rules.

XII.7 Transition to Result Formats

With the canonical state model (Section XI) and the epistemic interface layer (Section XII) now defined, the remaining question concerns output: how are the results of Alexandria's processes formally presented?

The following section describes Alexandria's result formats – without authority, but with maximum clarity.

Section XIII – Result Formats and Epistemic Status

XIII.1 Why Result Formats Are Decisive

A system that produces assessability rather than truth stands or falls by the nature of its outputs.

Unclear or unified results would: simulate authority, suggest consensus, glorify technology as truth.

Alexandria therefore treats result formats not as a side matter, but as an epistemic safety feature.

XIII.2 No Result in the Singular

Alexandria never delivers a single result.

Every output is multi-part and mandatorily contains:

- consistent propositions
- explicit assumptions
- remaining dissent

– formal undecidability (where applicable)
– identified evidence needs

A result without dissent is suspect.

XIII.3 Consistent Propositions

Propositions count as consistent only: relative to clearly named assumptions, under explicit derivation rules, without category mixing.

Consistency does not mean truth, but formal tenability.

Every consistent proposition is therefore: conditional, time-bound, and open to revision.

XIII.4 Explicit Assumptions

All assumptions are: fully listed, unambiguously categorized, individually referenceable.

Implicit assumptions count as a formal error. What is not explicit cannot be assessed.

XIII.5 Remaining Dissent

Dissent is not minimized or resolved, but visibly preserved.

Remaining dissent is classified as:

– assumption conflict
– conceptual divergence
– evidence gap
– methodological difference

Alexandria does not evaluate dissent. It makes it legible.

XIII.6 Formal Undecidability

Undecidability is a valid terminal state.

It obtains when: relevant evidence is missing, assumptions are unverifiable, or contradictory but consistent paths exist.

“Undecidable” does not mean “arbitrary,” but “currently not formally justifiable.”

XIII.7 Evidence Needs

Alexandria explicitly names: which data are missing, which observations could resolve the dissent, which assumptions would be verifiable.

This makes research more targeted, without steering it.

XIII.8 Temporal Versioning

Every result is: dated, versioned, historically embedded.

No result may be understood as: timeless, final, or concluded.

XIII.9 Transition to Epistemic Value

The described result formats are formally neutral. Before turning to societal effects, one further structural question arises: can the protocol’s own metrics be used to define a measurable notion of epistemic contribution?

The following section introduces epistemic value as a protocol-internal quantity – strictly structural, independent of truth claims, and explicitly separated from any economic implementation.

Section XIV – Epistemic Value and Optional Economic Layer Integration

XIV.1 Definition of Epistemic Value

The Alexandria Protocol defines structural properties of epistemic states, including uncertainty, stability, and temporal persistence.

These structural properties permit the definition of epistemic value as a measurable protocol-internal quantity.

Epistemic value is defined as the measurable reduction in epistemic uncertainty associated with a specific knowledge element over time.

Let $\sigma_k(t)$ denote the uncertainty associated with epistemic element k at time t.

Epistemic value contribution is defined as:

$V_k = \int \text{from } t_0 \text{ to } t_1 \text{ of } (\sigma_k(t_0) - \sigma_k(t))\, dt$

This represents the cumulative reduction in epistemic uncertainty over a defined interval.

Epistemic value reflects structural precision improvement. It does not represent correctness, truth probability, or usefulness.

XIV.2 Attribution of Epistemic Value

Epistemic value is attributed through patch lineage. Each patch modifies the epistemic state graph.

Each patch P_i contributes an incremental epistemic value:

$\Delta V_i = \sigma_k(t_{i-1}) - \sigma_k(t_i)$

Patch lineage provides complete attribution traceability. This ensures that epistemic contributions remain structurally attributable over time.

Attribution is independent of institutional identity or authority. Attribution depends exclusively on structural contribution.

XIV.3 Epistemic Value Persistence

Epistemic value persists as part of epistemic lineage. Value attribution remains valid even if the epistemic element is later deprecated.

Deprecation reflects structural invalidation, not retroactive erasure. Epistemic contributions remain permanently recorded.

This preserves historical epistemic accountability.

XIV.4 Separation of Epistemic Value and Economic Compensation

The Alexandria Protocol defines epistemic value metrics. It does not define economic compensation mechanisms.

The protocol SHALL NOT:

- assign monetary value
- enforce compensation

– define economic policies
– require economic incentives

Economic compensation MAY be implemented by external systems. Such systems MAY use epistemic value metrics as input.

Economic implementation remains outside protocol scope. This preserves protocol neutrality.

XIV.5 Compatibility with External Economic Systems

External systems MAY use epistemic value metrics to implement compensation mechanisms. Such systems include, but are not limited to:

– research funding allocation systems
– distributed knowledge markets
– decentralized economic coordination systems
– reputation systems

These systems SHALL operate independently of protocol validity. Protocol validity SHALL NOT depend on economic implementation. Epistemic admissibility remains structurally defined.

XIV.6 Dual-Layer System Compatibility

The Dual-Layer System (DLS) defines an optional economic architecture compatible with Alexandria.

In the DLS framework:

– the Structural Layer (SF-layer) produces epistemic patches
– the Realization Layer (RE-layer) consumes epistemically admissible knowledge

Epistemic value metrics MAY be used to compensate contributors in the SF-layer. Such compensation SHALL depend on externally defined economic rules.

Alexandria SHALL NOT enforce economic transfer. Alexandria SHALL provide only epistemic value measurement.

XIV.7 Epistemic Value Independence from Truth

Epistemic value measures reduction in structural uncertainty. Epistemic value does not guarantee correctness.

An epistemic contribution MAY reduce uncertainty while remaining incorrect. Subsequent patches MAY invalidate prior contributions.

Epistemic value reflects structural contribution to epistemic refinement. It does not certify correctness.

XIV.8 Economic Layer Independence Invariant

The Alexandria Protocol SHALL remain fully functional without any economic layer.

Protocol operation SHALL NOT depend on: monetary incentives, compensation mechanisms, or external economic systems.

Economic integration is optional. Protocol integrity remains independent of economic implementation.

XIV.9 Summary and Transition

The Alexandria Protocol introduces a measurable structural definition of epistemic value based on uncertainty reduction and epistemic lineage. This enables compatibility with external economic systems without compromising protocol neutrality.

Alexandria measures epistemic contribution. It does not assign economic reward. This preserves structural epistemic integrity while enabling optional economic integration.

The following section turns from architecture to impact: what societal effects and conflict lines does this system produce?

Section XV – Sociological Effects and Conflict Potential

XV.1 Why Social Effects Are Part of the Architecture

An epistemic system that reorders power, authority, and truth does not operate neutrally, even if it is formally neutral.

Alexandria changes not content, but the social mechanisms through which content acquires validity.

These effects are not side consequences, but systemic outcomes of the architecture.

XV.2 Power Shift: From Speaker to Process

Alexandria shifts epistemic power:

– away from persons
– away from institutions
– away from reputation

toward:

– formal processes
– reconstructible paths
– explicit assumptions

It is no longer the speaker who legitimizes the proposition, but the traceability of its path.

This relieves individual actors, but undermines existing power positions.

XV.3 Devaluation of Epistemic Authority

Titles, institutions, and historical status lose: their role as an epistemic shortcut and their protective function against formal critique.

This particularly affects:

– established scientific enterprises
– opinionated experts
– morally charged discourses
– media intermediaries

The expected resistance is structural, not personal.

XV.4 The Category Conflict

Alexandria's strict separation of: empirical, normative, model-based, and speculative stands in open conflict with disciplines and discourses that deliberately blend these levels.

In such contexts Alexandria will be perceived as: cold, reductionist, "depoliticizing."

In reality it is: demoralized, not depoliticized.

XV.5 Epistemic Burden of Proof

The obligation to complete reconstruction produces: higher effort, increased transparency, reduced rhetorical latitude.

This can lead to: defensive argumentation, restraint in speculation, initial self-censorship.

Alexandria does not prohibit speculation, but compels its explicit labeling.

XV.6 Consistency Misunderstandings

A central risk is that formal consistency is mistaken for truth.

Despite all self-binding: people seek certainty, consistency feels reassuring, technology is projected as authority.

Alexandria cannot eliminate this risk, only visibly contain it.

XV.7 Simultaneous Acceleration and Deceleration

Alexandria acts paradoxically: it accelerates the clarification of structural questions, while decelerating premature conclusions.

This collides with: media logic, political urgency, moral mobilization.

Conflicts are unavoidable.

XV.8 Epistemic Resilience

In the long term, Alexandria produces: greater robustness against manipulation, reduced dependence on trust, stronger separation of knowledge and power.

This is not a harmonious process, but a structural transformation.

XV.9 No Promise of Salvation

Alexandria promises: no better society, no fairer decisions, no conflict-free truth.

It promises only: that epistemic claims to power can no longer remain invisible.

XV.10 Transition to Demarcation

Precisely because of these effects, clear demarcation is necessary.

The following section therefore describes what Alexandria expressly is not – in order to prevent misprojections.

Section XVI – Demarcation, Non-Goals, and Misprojections

XVI.1 Why Demarcation Is Necessary

The more fundamental an architecture, the more heavily it is burdened with alien expectations.

Alexandria is particularly susceptible to projections because it operates at a sensitive interface: knowledge, truth, power.

Without explicit demarcation, Alexandria would be: instrumentalized for political purposes, misunderstood as a moral project, misinterpreted as technocratic authority.

This demarcation is therefore constitutive, not communicative.

XVI.2 What Alexandria Expressly Is Not

Alexandria is not:

- a ministry of truth
- a consensus generator
- a peer-review replacement
- a rating or ranking system
- a political decision-making body
- a moral corrective
- a governance model for societies
- a censorship or moderation tool

Each of these attributions contradicts the self-binding of the system.

XVI.3 No Substitute for Human Responsibility

Alexandria does not relieve people of responsibility.

It does not decide: what should be done, which risks are acceptable, which values should apply.

Alexandria structures judgments. Responsibility remains human.

The displacement of responsibility onto the system counts as misuse.

XVI.4 No Neutralization of Conflicts

Alexandria does not aim at: pacification, harmony, or balance.

Conflicts persist, but become formally visible.

Alexandria does not resolve conflicts. It only prevents them from masquerading as truth.

XVI.5 No Guarantee Against Misuse

Alexandria can be misused: selective use of results, rhetorical inflation of formal consistency, political instrumentalization.

The system does not protect against misuse, but against invisible misuse.

XVI.6 No Epistemic Leveling

Alexandria does not claim: that all propositions are equivalent, that expertise is irrelevant, that knowledge is relative.

It claims only: expertise operates not through rank, but through reconstructible derivation.

XVI.7 Protection Against Messianic Promises

Alexandria rejects all salvific rhetoric.

It promises: no better truth, no moral purification, no objective society.

A system that makes such promises would itself be ideological.

XVI.8 Transition to the Concluding Framework

After self-binding, architecture, effects, and demarcation, one final question remains:

How does Alexandria position itself in relation to existing system architectures and future developments?

The concluding section provides a formal answer.

Section XVII – Relation to Existing Architectures and Outlook

XVII.1 Positioning Within the Existing Knowledge Landscape

Alexandria does not aim to replace existing systems of knowledge production. It positions itself athwart them.

- Scientific disciplines remain sites of knowledge production.
- Institutions remain sites of legitimation and decision.
- The public remains the site of debate.

Alexandria inserts itself as epistemic infrastructure that does not steer these domains, but makes them transparent.

XVII.2 Relation to Science and Research

Alexandria is not an alternative science system.

It does not intervene in: method selection, research questions, or result evaluation.

It merely provides an additional, formal level: the level of verifiable derivation.

Scientific freedom remains untouched. Scientific authority is not abolished, but made formalizable.

XVII.3 Relation to Politics and Decision Systems

Alexandria makes no decisions and gives no recommendations.

Political systems may: consider Alexandria results, ignore them, or use them selectively.

Alexandria claims no steering function. It separates knowledge from decision, without devaluing either level.

XVII.4 Relation to the Public and Media

Alexandria is not optimized for audiences.

Its results are: in need of explanation, ambiguous, often unsatisfying in the sense of clear answers.

This places it in tension with: media abbreviation, dramatization, moral polarization.

This tension is not resolvable.

XVII.5 Relation to Other System Architectures

Alexandria is deliberately compatible, not exclusive.

It can: coexist with existing peer-review procedures, be integrated into scientific publications, serve as a supplementary verification layer.

It claims no monopoly.

XVII.6 Relation to the Dual-Layer System (DLS)

Alexandria and the Dual-Layer System operate on different levels:

- Alexandria: epistemic architecture
- DLS: economic architecture

Both share: moral neutrality, actor-independence, focus on structure rather than behavior.

Alexandria clarifies how knowledge is assessed. DLS clarifies how value flows are kept stable.

Both systems are independent, but complementary.

Alexandria further addresses a structural limitation of the current academic system: the citation pyramid as a mechanism of credibility and career advancement. Under the current system, epistemic authority accumulates through citation chains that are unverifiable, retractable, and susceptible to manipulation. Alexandria replaces this mechanism with cryptographically anchored contribution lineage. Each epistemic contribution is permanently and unambiguously attributable, tamper-proof, and reconstructible. This is not merely a technical convenience: it is a prerequisite for any future monetization of knowledge graphs. If epistemic contributions are to carry economic value – as anticipated in Section XIV – then manipulation of contribution records must be structurally excluded, not merely discouraged. The append-only hash chain provides precisely this guarantee.

Knowledge produced in the SF-Layer and formally validated through Alexandria can be transferred via the controlled interface as a public good to the RE-Layer. Compensation for knowledge

production remains in the SF-Layer, while the result – verified and category-pure – is made available to the real economy without additional ecological burden.

XV.7 Development Principles

Alexandria is not a finished product.

Its further development is subject to the same principles as its subject matter:

– versioning
– explicit assumptions
– documented dissent
– no authority shortcuts

Alexandria can be credible only if it develops itself according to its own rules.

Computational scalability:

Alexandria does not require global recomputation. Validation occurs locally at patch level. Complexity therefore scales with patch graph locality, not total epistemic state size. This ensures that the protocol remains computationally tractable as the epistemic state grows.

XV.8 Closing Remark

Alexandria promises: no truth, no agreement, no reassurance.

It promises only:

that epistemic paths remain visible,

that dissent does not vanish,

and that power can no longer conceal its claims to truth.

Nothing more. But nothing less.

Appendix A – Threat Model and Adversarial Robustness

A.1 Purpose

This appendix defines the adversarial threat model under which the Alexandria Protocol operates.

Alexandria assumes adversarial participation as a normal condition, not an exceptional case.

The protocol is designed to preserve epistemic integrity under conditions of: malicious participation, coordinated manipulation attempts, incomplete information, and adversarial model behavior.

Alexandria does not assume trusted participants. Alexandria assumes only structural protocol compliance.

A.2 Threat Model Scope

Let $\mathscr{A}$ denote the set of adversarial actors.

Adversarial actors MAY attempt to:

- submit misleading patches
- omit critical assumptions
- manipulate validation signals
- coordinate submission strategies
- attempt epistemic history alteration

The protocol MUST remain structurally robust under arbitrary adversarial participation.

Alexandria assumes adversaries have full protocol visibility. Security does not rely on secrecy.

A.3 Threat Class I – Malicious Patch Submission

Adversarial actors MAY submit structurally valid but misleading epistemic patches. Such patches MAY contain: incomplete assumptions, incorrect derivations, or misleading empirical claims.

Alexandria does not attempt to prevent submission of incorrect patches. Instead, Alexandria guarantees: incorrect patches remain detectable, contestable, and structurally bounded.

Mitigation mechanisms include:

- audit gate validation
- adversarial cross-assessment
- dissent branch formation
- stability decay over time

Incorrect patches cannot overwrite existing epistemic history. They can only create additional epistemic branches.

A.4 Threat Class II – Validation Manipulation Attempts

Adversarial actors MAY attempt to manipulate validation signals. This includes: coordinated validation submissions, artificial inflation of stability metrics, and selective validation omission.

Alexandria mitigates this through non-aggregative validation persistence measurement. Stability is defined as validation survival over time, not instantaneous validation volume.

Transient validation spikes do not produce permanent stability increases. Stability requires temporal persistence.

This prevents short-term validation manipulation from producing long-term epistemic authority.

A.5 Threat Class III – Epistemic History Tampering

Adversarial actors MAY attempt to alter prior epistemic states. This includes attempts to: modify prior patches, alter validation records, or erase dissent.

Alexandria prevents such tampering through cryptographic chain anchoring.

$C_i = Hash(P_i, C_{i-1})$

Any modification of prior patches produces hash inconsistency. This ensures tamper detectability of epistemic history. Epistemic history is immutable once committed.

A.6 Threat Class IV – Sybil Attacks

Adversarial actors MAY create multiple identities to simulate validation consensus.

Alexandria mitigates Sybil attacks through structural validation persistence. Validation identity does not directly determine stability.

Stability depends on:

- independent validation persistence
- cross-model reconstruction
- temporal survival under adversarial audit

Identity multiplication without structural validation does not produce stability.

A.7 Threat Class V – Model-Level Adversarial Behavior

Validation models themselves MAY behave adversarially. This includes: malicious validation approval, malicious rejection, and coordinated adversarial model behavior.

Alexandria mitigates this through multi-model cross-assessment. Dissent produces branches, not system collapse. No single model can unilaterally determine epistemic admissibility.

Epistemic state evolves through structural interaction, not centralized authority.

A.8 Threat Class VI – Consensus Capture Attempts

Adversarial actors MAY attempt to produce artificial epistemic consensus.

Alexandria explicitly does not use consensus aggregation as an admissibility mechanism. Admissibility depends on reconstructibility and structural validity, not consensus.

Consensus without reconstructibility has no epistemic effect. This prevents consensus capture from producing epistemic authority.

A.9 Threat Class VII – Retroactive Narrative Manipulation

Adversarial actors MAY attempt to retroactively reinterpret epistemic history.

Alexandria prevents this through append-only patch chains. Prior epistemic states remain permanently visible. Deprecation marks epistemic invalidation without erasing historical structure.

This ensures epistemic transparency over time.

A.10 Explicit Non-Protection Guarantees

Alexandria does not guarantee protection against:

- submission of incorrect claims
- temporary epistemic confusion
- malicious participant intent
- adversarial epistemic competition

Alexandria guarantees only:

– structural reconstructibility
– epistemic lineage preservation
– tamper detection
– dissent visibility

Alexandria preserves epistemic structure. It does not enforce epistemic correctness.

A.11 Security Model Summary

Under adversarial participation, the following invariant holds:

Integrity(E(t)) = Reconstructibility + Immutability + Structural Transparency

This invariant holds independent of adversarial participation level.

Epistemic correctness remains outside protocol scope. Epistemic integrity remains guaranteed.

Appendix B – End-to-End Epistemic Lifecycle Examples

B.1 Purpose

This appendix illustrates the operational behavior of the Alexandria Protocol through complete epistemic lifecycles.

These examples are non-normative. They do not define protocol rules. They demonstrate protocol execution under realistic epistemic conditions.

These examples illustrate structural protocol behavior. They do not imply technological feasibility, economic viability, or scientific correctness of the underlying material claim.

B.2 Example I – Initial Discovery and Structural Seal

B.2.1 Discovery Submission

A research team discovers a solid-state battery anode material which increases measured energy density. They submit the claim via Patch-DSL.

PATCH {

```
        patch_id: "solid-state-anode-001"
        parent_patch_id: "energy-storage-tree-41"
        branch_id: "b0"
        target_type: "NODE"
        target_id: "Energy_Storage_Tech_Tree_Node_42"
        timestamp: 1771459200
        operation: "ADD"
        category: "EMPIRICAL"
        derivation {
          type: "EMPIRICAL"
          method_reference: "LAB_REPLICATION_PROTOCOL_V7"
        }
        assumptions: [
          "Temp_Range_18C_25C",
          "Pressure_1atm",
          "Measurement_Method_Calibrated"
        ]
        validation {
          validation_function_ref: "VAL_EMPIRICAL_REPLICATION"
          stability_index: 0.62
          stability_parameters {
            decay_constant: 0.00038
          }
          cross_assessment_ref: "AUDIT_SOLID_STATE_001"
        }
        uncertainty {
          type: "PROBABILISTIC"
          sigma: 0.04
          confidence_interval: [0.16, 0.24]
          sample_size: 1200
        }
}
```


Initial state transition: ∅ → UNVALIDATED

B.2.2 Audit and Validation

Audit checklist passes all structural requirements.

State transition: UNVALIDATED → VALIDATED

B.2.3 Temporal Stability Accumulation

Independent replication increases stability:

$S(t_0) = 0.62 \rightarrow S(t_1) = 0.74 \rightarrow S(t_2) = 0.83$

B.2.4 Structural Seal

Seal patch:

```
PATCH {
        patch_id: "solid-state-anode-001-seal"
        parent_patch_id: "solid-state-anode-001"
        operation: "MODIFY"
        seal {
          seal_type: "STRUCTURAL_SEAL"
          sealed_at: 1774000000
        }
}
```

State transition: VALIDATED → SEALED

B.3 Example II – Adversarial Dissent and Branch Formation

B.3.1 Detection of Boundary Condition Failure

Adversarial model detects instability outside original temperature range. Branch patch emitted:

PATCH {
    patch_id: "solid-state-anode-dissent-001"
    parent_patch_id: "solid-state-anode-001"
    branch_id: "b_temp_constraint"
    operation: "BRANCH"
    category: "MODEL"
    derivation {
      type: "SIMULATION"
      method_reference: "SIM_LOG_ALPHA_7"
    }
    assumptions: [
      "Thermal_Stability_Model_v2"
    ]
    validation {
      validation_function_ref: "VAL_SIMULATION_STRESS"
      stability_index: 0.41
      stability_parameters {
        decay_constant: 0.00038
      }
      cross_assessment_ref: "AUDIT_DISSENT_ALPHA_7"
    }
}

State transition: SEALED → STABLE_AMBIGUITY

Both branches remain valid under their respective assumptions.

B.3.2 Epistemic Status Report

System now reports:

Status: STABLE_AMBIGUITY

Branch A: Valid under assumption set A

Branch B: Valid under assumption set B

No branch is erased. No forced consensus occurs.

B.4 Example III – Global Refutation and Deprecation

B.4.1 Detection of Measurement Artifact

Independent audit discovers systematic measurement calibration drift. Corrective patch:

```
PATCH {
    patch_id: "solid-state-anode-artifact-detection"
    parent_patch_id: "solid-state-anode-001"
    operation: "MODIFY"
    category: "EMPIRICAL"
    derivation {
      type: "EMPIRICAL"
      method_reference: "CALIBRATION_VERIFICATION_PROTOCOL_V3"
    }
    assumptions: [
      "Instrument_Drift_Model"
    ]
    validation {
      validation_function_ref: "VAL_EMPIRICAL_REPLICATION"
      stability_index: 0.91
      stability_parameters {
        decay_constant: 0.00025
      }
    }
}
```

B.4.2 Dependency-Driven Deprecation

Deprecation patch emitted:

```
PATCH {
```

```
    patch_id: "solid-state-anode-deprecate"
    operation: "DEPRECATE"
    target_id: "Energy_Storage_Tech_Tree_Node_42"
}
```

State transition: STABLE_AMBIGUITY → DEPRECATED

B.4.3 Epistemic Status Report

Status: DEPRECATED

Reason: Foundational measurement assumption invalidated.

Historical structure preserved. Prior epistemic states remain visible and reconstructible.

B.5 Protocol Behavior Demonstrated

These examples demonstrate the following protocol invariants:

- Epistemic evolution occurs through explicit patches
- Dissent results in branching, not erasure
- Refutation results in deprecation, not deletion
- Epistemic history remains reconstructible
- Structural admissibility is distinct from correctness

B.6 Protocol Self-Binding Preserved

At no point does the protocol: declare truth, enforce consensus, or erase prior epistemic structure.

The protocol preserves structure, not correctness.

B.7 Reconstruction Guarantee Demonstration

Given the patch sequence $P_1, P_2, \ldots, P_n$, the epistemic state at time t_n is uniquely reconstructible:

$E(t_n) = \mathscr{P}_n(\ldots\mathscr{P}_2(\mathscr{P}_1(E_0))\ldots)$

This property is guaranteed by blockchain anchoring and patch immutability.

Appendix C – Formal Guarantees and Structural Invariants

C.1 Purpose

This appendix defines the formal guarantees and invariants of the Alexandria Protocol.

These guarantees follow directly from the structural definitions of: patch-based epistemic state transitions, append-only persistence, explicit assumption declaration, and adversarial audit structure.

These guarantees are structural. They do not depend on correctness of epistemic claims. They depend only on protocol compliance.

C.2 State Transition Determinism

Let E(t) denote epistemic state at time t, and Δ(E(t), ⊠, t) denote a valid patch application.

Then:

$E(t^{+}) = \Delta(E(t), \Delta, t)$

State transitions are deterministic. Given identical prior state and patch, the resulting epistemic state is uniquely determined.

This ensures epistemic state reproducibility.

C.3 Epistemic Lineage Preservation Guarantee

Let $P_1, P_2, \ldots, P_n$ denote the patch sequence. Then epistemic state is defined as:

$E(t_n) = \mathscr{P}_n(\mathscr{P}_{n-1}(\ldots\mathscr{P}_1(E(t_0))\ldots))$

Because patches are immutable and append-only, prior epistemic states remain permanently reconstructible.

No epistemic transition erases prior epistemic structure. This ensures complete epistemic lineage preservation.

C.4 Tamper Detectability Guarantee

Each patch is cryptographically anchored:

$C_i = Hash(P_i, C_{i-1})$

Any modification of prior patch content results in chain hash mismatch. Therefore: epistemic tampering is cryptographically detectable.

The protocol guarantees tamper detectability. The protocol does not require trusted participants.

C.5 Explicit Assumption Guarantee

Protocol compliance requires explicit assumption declaration. Let A_k denote the assumption set for epistemic element k.

All derivations SHALL explicitly reference assumptions. Therefore: implicit assumptions constitute a protocol violation.

This ensures epistemic transparency of derivation dependencies.

C.6 Branch Preservation Guarantee

Let dissent occur at time t_d. Protocol rules require branch formation:

$G(t_d^{+}) = Branch(G(t_d))$

Branch formation preserves both epistemic paths. No dissent path is erased.

This ensures epistemic dissent preservation.

C.7 Temporal Ordering Guarantee

Each patch SHALL include a timestamp t_i. State transitions must satisfy:

$t_{i+1} > t_i$

Retroactive state modification is prohibited. This ensures forward-only epistemic evolution.

C.8 Stability as Validation Persistence Guarantee

Stability $S_k(t)$ is defined as validation persistence:

$S_k(t) = \int \text{from } t_0 \text{ to } t \text{ of } v_k(\tau) \cdot e^{(-\lambda_k(t-\tau))}\, d\tau \,/\, \int \text{from } t_0 \text{ to } t \text{ of } e^{(-\lambda_k(t-\tau))}\, d\tau$

This measures validation persistence, not correctness. Stability reflects structural survival under validation. Stability does not certify truth.

C.9 Epistemic Identity Invariant

Each epistemic element possesses persistent identity:

$ID(k) = Hash(initial_patch, lineage)$

Identity remains invariant across time. Branching creates new identities derived from parent lineage.

This ensures unambiguous epistemic reference.

C.10 Epistemic Value Attribution Guarantee

Let $\sigma_k(t)$ denote epistemic uncertainty. Epistemic value contribution is:

$\Delta V_i = \sigma_k(t_{i-1}) - \sigma_k(t_i)$

Patch lineage guarantees attribution traceability. Epistemic contributions remain permanently attributable.

C.11 Adversarial Robustness Guarantee

Under adversarial participation, protocol invariants remain preserved.

Adversarial actors MAY submit incorrect patches and attempt manipulation. However, adversarial actions cannot:

- erase prior epistemic structure
- retroactively modify epistemic lineage
- conceal dissent

Protocol guarantees structural integrity under adversarial conditions.

C.12 Non-Guarantees (Explicit Self-Binding)

The Alexandria Protocol does NOT guarantee:

– epistemic correctness
– scientific truth
– predictive accuracy
– consensus

The protocol guarantees only:

– reconstructibility
– lineage preservation
– explicit assumptions
– tamper detectability
– dissent preservation

This preserves epistemic neutrality.

Non-completeness:

The Alexandria Protocol does not guarantee that all epistemically valid knowledge will be discovered. It guarantees only that submitted epistemic paths remain structurally reconstructible.

Model fallibility:

Validation models themselves are epistemic participants subject to the same structural limitations as any other agent. No model possesses privileged epistemic access. Model outputs are formal contributions, not authoritative determinations.

Computational locality:

The protocol does not require global graph evaluation. Validation occurs locally at patch lineage level. Computational complexity scales with local patch graph structure, not total epistemic state size.

C.13 Structural Integrity Theorem

Let protocol compliance hold. Then epistemic state satisfies:

Integrity(E(t)) = Reconstructibility + Immutability + Transparency + Lineage Preservation

This invariant holds independent of epistemic correctness.

Protocol guarantees structural epistemic integrity.

Appendix D – Minimal Reference Implementation Sketch

D.1 Purpose

This appendix demonstrates the operational constructibility of the Alexandria Protocol through a minimal reference implementation sketch.

The purpose is not to provide a production implementation. The purpose is to demonstrate that the protocol can be instantiated using standard computational primitives.

This appendix illustrates: epistemic graph construction, patch emission, audit validation, branch formation, and epistemic state reconstruction.

D.2 Minimal Epistemic Graph Representation

A minimal epistemic graph can be represented as:

```
EpistemicGraph = {
    "nodes": {},
    "patch_chain": [],
    "branches": {}
}
```

Each epistemic element is represented as a node:

```
Node = {
    "id": "node_id",
    "content": "...",
    "category": "EMPIRICAL",
    "assumptions": [],
    "uncertainty": { "sigma": 0.2, "confidence_interval": [0.6, 0.8] },
    "stability": 0.0
}
```

D.3 Minimal Patch Structure

A minimal patch is represented as:

```
Patch = {
    "patch_id": "patch_001",
    "parent_patch": None,
    "operation": "ADD",
    "target_node": "node_001",
    "timestamp": 1771459200,
    "content": Node,
    "hash": None
}
```

Patch hash computation:

```
import hashlib, json

def compute_hash(patch):
    return hashlib.sha256(
        json.dumps(patch, sort_keys=True).encode()
    ).hexdigest()
```

D.4 Patch Application Function

Patch application is deterministic:

```
def apply_patch(graph, patch):
    node = patch["content"]
    graph["nodes"][node["id"]] = node
    graph["patch_chain"].append(patch)
```

This implements the protocol operator:

$E(t^+) = (\mathcal{P}(t), \Delta, t)$

D.5 Minimal Audit Gate

Audit gate verifies structural admissibility:

```
def audit_patch(patch):
    required = ["patch_id", "operation", "target_node", "timestamp", "content"]
    for field in required:
        if field not in patch: return False
    node = patch["content"]
    if node["category"] not in ["EMPIRICAL", "MODEL", "NORMATIVE", "SPECULATIVE"]:
        return False
    return True
```

Only structurally valid patches are applied.

D.6 Minimal Stability Update Function

Stability is updated based on validation persistence:

```
def update_stability(node, validated=True, decay=0.01):
    if validated:
        node["stability"] += (1 - node["stability"]) * 0.1
    node["stability"] *= (1 - decay)
```

This approximates:

$S_k(t) = \int \text{from } t_0 \text{ to } t \text{ of } v_k(\tau) \cdot e^{(-\lambda_k(t-\tau))}\, d\tau \,/\, \int \text{from } t_0 \text{ to } t \text{ of } e^{(-\lambda_k(t-\tau))}\, d\tau$

D.7 Branch Formation

Branch creation preserves dissent:

```
def create_branch(graph, parent_patch_id, branch_id):
    graph["branches"][branch_id] = {
        "parent": parent_patch_id,
        "patch_chain": []
    }
```

Branches preserve epistemic multiplicity. No branch overwrites another.

D.8 Epistemic State Reconstruction

Epistemic state is reconstructible from patch chain:

```
def reconstruct_state(patch_chain):
    graph = { "nodes": {}, "patch_chain": [], "branches": {} }
    for patch in patch_chain:
        apply_patch(graph, patch)
    return graph
```

This guarantees the reconstructibility invariant.

D.9 Minimal End-to-End Example

Step 1 – Create node:

```
node = {
    "id": "claim_001",
    "content": "Material X increases energy density",
    "category": "EMPIRICAL",
    "assumptions": ["measurement_protocol_v1"],
    "uncertainty": { "sigma": 0.2 },
    "stability": 0.0
}
```

Step 2 – Create patch:

```
patch = {
    "patch_id": "patch_001",
    "parent_patch": None,
    "operation": "ADD",
    "target_node": "claim_001",
    "timestamp": 1771459200,
    "content": node
}
patch["hash"] = compute_hash(patch)
```

Step 3 – Audit:

```
if audit_patch(patch):
    apply_patch(graph, patch)
```

Step 4 – Reconstruct:

```
graph = reconstruct_state(graph["patch_chain"])
```

Epistemic state is fully reconstructible.

D.10 Implementation Implications

This minimal sketch demonstrates:

– protocol constructibility
– deterministic state evolution
– audit gate enforcement
– branch preservation
– reconstructibility guarantee

Full implementations may extend this structure. This minimal implementation satisfies protocol invariants.

A complete, executable reference implementation (alexandria_core.py) accompanies this document. It implements the full operational pipeline including hash-chain anchoring, tamper detection during reconstruction, branch management, and stability decay – in approximately 300 lines of standard Python with no external dependencies.

Anhang A — Canonical Graph Schema (v2)

Der Alexandria-Graph basiert auf dem kanonischen Property-Graph-Modell des Protokolls. Alle Builder-Ergebnisse müssen in dieses Schema überführt werden, bevor Diff-Analysen durchgeführt werden.

Korrekturen in Anhang A (Protokollreferenz: Section VII.13.2, XI.2, X.3)
1. Alle Knotentypen erhalten ein assumptions[]-Feld (Protokollpflicht: Section VII.13.2).
2. Der Node Claim erhält uncertainty { sigma, ci, n, type } statt confidence (Section VII.3/VII.4).
3. Der Node Claim erhält lineage[] und validation{} (Protokollpflicht: Section XI.2, VII.13.6).
4. Der Node Claim erhält category als eigenes Feld, getrennt von modality (Section III.3).
5. Statuswerte des Claim-Knotens werden auf Protokoll-Statusklassen gemappt (Section X.8).
6. Die Validierungs-Relationen erhalten PATCH_ANCHORS_TO als neue Relation (Section VIII.7).

A.1 Grundprinzipien

- Jeder Knoten besitzt eine stabile globale ID (Epistemic Identity: Hash(initial_patch, lineage)).
- Jede Relation ist typisiert und gerichtet.
- Jede Aussage über die Welt wird als Claim-Knoten modelliert.
- Jeder Claim besitzt eine Quellenkette und explizite Annahmen.
- Jeder Claim besitzt einen epistemischen Status gemäß Protokoll-Statusklassen.
- Uncertainty wird nie als Einzelwert gespeichert, sondern als Tuple (sigma, ci, n).

A.2 Knotentypen

Entity

```
Node: Entity
```

entity_id	string (stable global ID)
label	string
aliases[]	string[]
entity_type	enum: Person | Institution | Technology | ...
description	string
assumptions[]	string[] // Protokollpflicht ◂
source_refs[]	ref: Work | external_uri
created_at	timestamp (ISO 8601, monoton)

Concept

Node: Concept	
concept_id	string
label	string
parent_concepts[]	ref: Concept[]
definition	string
ontology_refs[]	string (OpenCyc URIs)
domain	string
assumptions[]	string[] // Protokollpflicht ◂
created_at	timestamp

Work

Node: Work	
work_id	string
title	string
authors[]	ref: Author[]

year	int
venue	string
doi	string
abstract	string
citation_count	int
openalex_id	string
created_at	timestamp

Author

Node: Author	
author_id	string
name	string
affiliations[]	ref: Institution[]
openalex_id	string
orcid	string
created_at	timestamp

Institution

Node: Institution	
institution_id	string
name	string
country	string
type	enum: University | ResearchInstitute | Company | ...
openalex_id	string
created_at	timestamp

Claim — Zentrale epistemische Einheit (mehrere Korrekturen)

Node: Claim (v2)	
claim_id	string (stable global ID)
subject	ref: Entity | Concept
predicate	ref: RelationType
object	ref: Entity | Concept | Claim
category	enum: EMPIRICAL | NORMATIVE | MODEL | SPECULATIVE // neu ◀
qualifiers	map<string, any>
scope	map<string, any>
time_scope	map<string, any>
modality	enum: hypothesis | suggestion | evidence | established
uncertainty	{ sigma, ci, n, type } // ersetzt confidence ◀
status	enum: UNVALIDATED | VALIDATED | SEALED | FORMAL_ERROR | ◀
	STABLE_AMBIGUITY | DEPRECATED | ◀
	partially_supported* | derived* (* DBA-Erweiterungen) ◀
assumptions[]	string[] // Protokollpflicht ◀
evidence_refs[]	ref: Evidence[]
source_refs[]	ref: Work[]
validation	{ validated, decay, refs[] } // neu ◀
builder_origin	enum: alpha | beta | adjudicated // DBA-Erweiterung
lineage[]	patch_id[] // Protokollpflicht ◀
created_at	timestamp (ISO 8601, monoton)
version	int

Evidence

Node: Evidence	
evidence_id	string
source_work	ref: Work
text_excerpt	string
page	int | null
uncertainty	{ sigma, ci, n, type } // war: confidence ◀
created_at	timestamp

Judgment

Node: Judgment	
judgment_id	string
diff_id	ref: Diff
rule_applied	ref: AdjudicationRule
decision	string
review_agent	string
timestamp	timestamp (ISO 8601, monoton)
blockchain_anchor	string // commit_hash, Protokollpflicht ◀
notes	string

A.3 Relationstypen

Ontologische Relationen

INSTANCE_OF	Entität ist Instanz eines Konzepts
SUBCLASS_OF	Konzept ist Unterklasse eines anderen Konzepts

`PART_OF`	Strukturelle Teilbeziehung
`RELATED_TO`	Allgemeine assoziative Relation (schwach, Auffangkategorie)

Wissenschaftliche Relationen

`AUTHORED_BY`	Work wurde von Author verfasst
`AFFILIATED_WITH`	Author ist mit Institution verbunden
`CITES`	Work zitiert ein anderes Work
`PUBLISHED_IN`	Work erschien in einem Venue

Claim-Relationen

`CLAIM_ABOUT`	Claim bezieht sich auf eine Entität oder ein Konzept
`CLAIM_RELATES_TO`	Claim verbindet zwei Knoten
`SUPPORTED_BY`	Claim wird durch Evidence oder Work gestützt
`CONTRADICTS`	Claim steht im Widerspruch zu einem anderen Claim
`REFINES`	Claim präzisiert einen anderen Claim (feinere Granularität)
`DERIVED_FROM`	Claim wurde aus einem anderen Claim abgeleitet

Evidenz-Relationen

`HAS_EVIDENCE`	Claim ist durch eine konkrete Evidence-Instanz belegt
`MENTIONED_IN`	Entität oder Konzept wird in einem Work erwähnt

Validierungs-Relationen

`ADJUDICATED_BY`	Claim oder Diff wurde durch ein Judgment verarbeitet
`GENERATED_BY`	Knoten wurde von einem bestimmten Builder erzeugt
`DIFF_WITH`	Verbindet zwei konkurrierende Knoten desselben Typs

PATCH_ANCHORS_TO ◂	Patch ist kryptographisch an Blockchain-Commit gebunden // neu

Anhang B — Diff Ontology (v2)

Die Diff-Ontology beschreibt alle möglichen strukturellen Differenzen zwischen zwei Builder-Graphen. Alle Diffs werden typisiert und erhalten eine eindeutige ID.

Korrekturen in Anhang B (Protokollreferenz: Section VII.3, VII.4, III.3)
1. Neuer Diff-Typ: assumption_mismatch (B.8) – Protokollpflicht für explizite Annahmen.
2. Neuer Diff-Typ: category_mismatch (B.9) – EMPIRICAL/NORMATIVE/MODEL/SPECULATIVE.
3. Umbenennung: confidence_divergence → uncertainty_divergence (spiegelt das Tuple-Modell).

B.1 Entity Diffs

missing_entity	Eine Entität existiert nur in einem Builder-Graph.
entity_alias_mismatch	Gleiche Entität, unterschiedliche Namensrepräsentationen.
entity_type_mismatch	Unterschiedliche Klassifikation einer Entität.

B.2 Concept Diffs

concept_mapping_mismatch	Unterschiedliche Zuordnung zum OpenCyc-Ontologiekonzept.
concept_hierarchy_mismatch	Unterschiedliche Position im Konzeptbaum.
concept_scope_mismatch	Begriff wird mit unterschiedlicher Breite interpretiert.

B.3 Claim Diffs

missing_claim	Claim existiert nur bei einem Builder.

`claim_granularity_mismatch`	Claim ist unterschiedlich detailliert formuliert.
`claim_scope_mismatch`	Unterschiedlicher Geltungsbereich.
`claim_modality_mismatch`	Unterschiedliche epistemische Stärke: suggests vs. supports vs. proves.
`claim_quantifier_mismatch`	Unterschiedliche Mengenquantoren.
`claim_temporal_mismatch`	Unterschiedlicher Zeitbezug.

B.4 Relation Diffs

`relation_mismatch`	Unterschiedlicher Relationstyp.
`relation_direction_mismatch`	Beziehungsrichtung unterschiedlich gesetzt.
`causality_mismatch`	Ein Builder liest Kausalität, der andere nur Korrelation.

B.5 Evidence Diffs

`source_link_mismatch`	Ähnliche Claims, aber an unterschiedliche Quellen angebunden.
`evidence_strength_mismatch`	Gleiche Quelle, aber unterschiedliche Evidenzbewertung.
`citation_scope_mismatch`	Ein Builder verallgemeinert aus einer Quelle zu stark.

B.6 Uncertainty Diffs ◂ (vorher: Confidence Diffs)

Umbenennung (Protokollreferenz: Section VII.3/VII.4)
confidence_divergence wird zu uncertainty_divergence, da das Feld jetzt ein Tuple (sigma, ci, n) ist,
kein Einzelwert. Die Schwellenwertlogik bleibt erhalten, bezieht sich nun aber auf sigma-Divergenz.

`uncertainty_divergence_minor ◂`	sigma-Unterschied unter Schwellenwert (z. B. < 0.15).

`uncertainty_divergence_major` ◂	sigma-Unterschied über Schwellenwert – erfordert Adjudication.
`uncertainty_type_mismatch` ◂	Ein Builder klassifiziert probabilistisch, der andere deterministisch.

B.7 Status Diffs

`status_mismatch`	Unterschiedlicher epistemischer Status. Kritischster Fall: z. B. VALIDATED vs. FORMAL_ERROR.

B.8 Assumption Diffs ◂ (neu)

Neuer Diff-Typ (Protokollreferenz: Section VII.13.2, X.4)
Das Protokoll verlangt explizite Annahmen als Pflichtfeld. Unterschiedliche Annahmenmengen
zwischen Alpha und Beta sind daher ein eigenständiger, typisierter Diff.

`assumption_mismatch` ◂	Unterschiedliche Annahmemengen zwischen Alpha und Beta.
`assumption_scope_mismatch` ◂	Gleiche Annahme, aber unterschiedliche Auslegung oder Granularität.

B.9 Category Diffs ◂ (neu)

Neuer Diff-Typ (Protokollreferenz: Section III.3, X.3)
Das Protokoll verlangt Kategorienpurität (EMPIRICAL / NORMATIVE / MODEL / SPECULATIVE).
Unterschiedliche Kategoriezuordnungen sind ein eigenständiger kritischer Diff-Typ.

`category_mismatch` ◂	Alpha und Beta ordnen denselben Claim unterschiedlichen Protokoll-Kategorien zu.

Anhang C — Adjudication Rulebook (v2)

Das Rulebook definiert, wie mit jeder Diff-Kategorie umzugehen ist. Grundsatz: keine Mittelung, keine künstliche Harmonisierung, keine Aggregation von Uncertainty.

Korrekturen in Anhang C (Protokollreferenz: Section VII.4, III.3, X.3)
1. Regelklasse C.6 (Confidence) wird zu Uncertainty-Regel (kein Einzelwert, sigma-Divergenz).
2. Neue Regelklasse C.8: Category Mismatch (Protokollpflicht: Kategorienpurität).
3. Neue Regelklasse C.9: Assumption Mismatch (Protokollpflicht: explizite Annahmen).

Regelklasse	Trigger	Ergebnis
C.1 Konservative Präzisierung	Zwei Claims unterschiedlich stark formuliert.	`Vorsichtigere Relation gewinnt. supports + partially_supports → partially_supported (DBA) bzw. STABLE_AMBIGUITY (Protokoll)`
C.2 Granularitäts-regel	Ein Claim detaillierter, textlich gestützt.	`Feinere Version wird übernommen. Grober Claim: DERIVED_FROM markiert.`
C.3 Konflikt-regel	Zwei Claims nicht vereinbar.	`status = STABLE_AMBIGUITY oder FORMAL_ERROR Konflikt bleibt sichtbar. [v2.2] Kein stiller Alpha-Default. Unbekannte Diff-Typen: UNRESOLVED_PENDING_RULE, winning_id = None.`
C.4 Oberbegriff-regel	Unterschiedliche Ontologiezuordnungen.	`Gemeinsamer Oberbegriff versiegeln. Spezialzuordnung offen lassen.`
C.5 Missing-Claim- Regel	Claim nur bei einem Builder.	`Prüfung: textlich gestützt? Implizit oder explizit? Redundanz? Überinterpretation? Erst danach Entscheidung.`
C.6 Uncertainty-Regel ◄	Große sigma-Divergenz zwischen Alpha und Beta.	`Kein Mitteln. uncertainty_divergence_major loggen. status = STABLE_AMBIGUITY Bei kleiner Divergenz: uncertainty_divergence_minor`
C.7 Relations-priorität	Builder uneinig über Relationstyp.	`Epistemisch defensivere Relation gewinnt. mentions → relates_to → correlates_with → partially_supports → supports → strongly_supports`

C.8 Category-Regel ◄	category_mismatch zwischen Alpha und Beta.	`Kategoriemischung = FORMAL_ERROR (Protokollpflicht Section X.3). Branch wenn nicht auflösbar. Kein Kompromiss zwischen Kategorien.`
C.9 Assumption-Regel ◄	assumption_mismatch oder fehlende Annahmen.	`Fehlende assumptions[] = UNVALIDATED. Assumption-Divergenz: explizit loggen. Bei unauflösbarem Konflikt: Branch.`

Anhang D — Seal Policy (v2)

Ein Alexandria-Snapshot wird nur freigegeben, wenn definierte Stabilitätsbedingungen erfüllt sind. Diese Version ergänzt die Kriterien um protokollkonforme Prüfpunkte.

Korrekturen in Anhang D (Protokollreferenz: Section X.3–X.8, XI.4, VIII.7)
1. D.3 ergänzt um category_mismatch und assumption_mismatch als kritische Diffs.
2. D.5 ergänzt: Judgment-Einträge müssen blockchain_anchor tragen.
3. Neues Kriterium D.9: Audit-Block-Vollständigkeit (alle 5 Blocks gemäß Section X).
4. D.7 (Versionierung) ergänzt um decay_config (lambda-Parameter müssen deklariert sein).

D.1 – D.9 Seal-Kriterien

D.1 Structural Integrity	Keine verwaisten Knoten. Keine ungültigen Relationen. Keine ungebundenen Claims. Alle lineage[]-Referenzen auflösbar.
D.2 Source Traceability	Jeder versiegelte Claim hat mindestens eine nachvollziehbare Quelle (ref: Work oder Evidence).
D.3 Critical Diff Resolution ◄ ◂	Folgende Diff-Typen müssen aufgelöst sein: relation_mismatch, causality_mismatch, concept_mapping_mismatch, status_mismatch, category_mismatch, assumption_mismatch // neu
D.4 Diff Budget	Begrenzte Anzahl nicht-kritischer Differenzen pro Subgraph (konfigurierbar).
D.5 Judgment Completeness ◄ ◂	[v2.2 Änderung] D.5 ist ADVISORY ONLY — nicht-blockierend (Sprint 1). Maturity (Φ) darf keinen formal korrekten Seal blockieren. Φ-Wert wird geloggt und im SealRecord dokumentiert. Maturity informiert operative Deployment-Entscheidungen, substituiert aber keine Seal-Kriterien D.1–

	D.4, D.6.
D.6 Stabilitäts-kriterium	Mehrere Iterationen: neue kritische Diffs pro Iteration unter definierter Schwelle.
D.7 Versionierung ◄ ◂	Jeder Snapshot trägt: snapshot_id, graph_hash, timestamp, builder_versions, ontology_version, diff_report_hash, decay_config // neu: lambda-Parameter müssen deklariert sein
D.8 Freigabe- status	Snapshots durchlaufen: candidate → validated → sealed → deprecated Zustände entsprechen Protokoll-Statusklassen (Section X.8).
D.9 Audit-Block-Vollständigkeit ◄ ◂	Alle 5 Audit-Blocks müssen bestanden sein (Section X.3–X.7): I: Category Purity II: Path Reconstruction III: Temporal Integrity IV: Cross-Assessment V: Uncertainty Disclosure

◄ **markierte Kriterien** sind Korrekturen oder Ergänzungen gegenüber v1. Alle anderen Kriterien waren bereits protokollkompatibel.

Ende der Anhänge A–D (v2)

Teil I — Korrekturen gegenüber v1

Korrektur 1 — confidence wird ersetzt durch uncertainty

Protokollreferenz: Section VII.3 / VII.4 (Alexandria Protocol v2.01)
Das Protokoll verbietet explizit, Unsicherheit in einen einzelnen Float-Wert zu kollabieren.
U_system = { $U_1(k)$, $U_2(k)$, …, $U_n(k)$ } – nicht ein gemitteltes $\bar{U}(k)$.
confidence = 0.73 verletzt dieses Prinzip. Builder können intern Confidence-Werte verwenden,
aber der versiegelte Claim muss das Uncertainty-Tuple tragen.

Alt (v1):

```
confidence      float [0.0 – 1.0]
```

Neu (v2):

```
uncertainty     map {
  sigma:        float,
  ci:           [float, float],    // Konfidenzintervall
  n:            int,               // Stichprobengröße
```

```
  type:         probabilistic | deterministic
}
// Für deterministische Claims: uncertainty.type = deterministic, sigma/ci/n
leer.
// Builder-interne Confidence wird NICHT in den versiegelten Claim
übernommen.
```

Korrektur 2 — Status-Taxonomie auf Protokoll-Statusklassen gemappt

Protokollreferenz: Section X.8 (Alexandria Protocol v2.01)
Das Protokoll definiert: UNVALIDATED, VALIDATED, SEALED, FORMAL_ERROR, STABLE_AMBIGUITY, DEPRECATED.
Die v1-Taxonomie war feiner, aber nicht direkt kompatibel. Zwei Werte haben kein Analogon
und werden als begründete Erweiterung der Dual-Builder-Architektur deklariert.

Anhang E (v1)	Alexandria-Protokoll	Hinweis
`supported`	`SEALED`	*Claim hat vollständige Validierung bestanden*
`partially_supported`	`(Erweiterung)`	*Konditionale Unterstützung – kein direktes Protokolläquivalent; begründete DBA-Erweiterung*
`disputed`	`STABLE_AMBIGUITY`	*Zwei konsistente, unvereinbare Pfade koexistieren*
`contradicted`	`FORMAL_ERROR`	*Strukturverletzung oder Widerlegung durch Evidenz*
`underspecified`	`UNVALIDATED`	*Claim noch nicht auditierbar*
`derived`	`(Erweiterung)`	*Aus anderen Claims abgeleitet – DBA-Erweiterung; kein direktes Protokolläquivalent*
`obsolete`	`DEPRECATED`	*Durch neuere Version ersetzt*

Korrektur 3 — category und modality sind orthogonale Felder

Protokollreferenz: Section III.3 (Alexandria Protocol v2.01)
Das Protokoll unterscheidet EMPIRICAL / NORMATIVE / MODEL / SPECULATIVE als Kategorien.

Modality (hypothesis / suggestion / evidence / established) ist eine orthogonale epistemische Stärke.
Ein EMPIRICAL-Claim kann hypothesis-Stärke haben. Ein MODEL-Claim kann established sein.
Beide Felder bleiben erhalten, dürfen aber nicht verwechselt oder zusammengeführt werden.

```
// Zwei getrennte Felder – keine implizite Abhängigkeit:
category        enum: EMPIRICAL | NORMATIVE | MODEL | SPECULATIVE
modality        enum: hypothesis | suggestion | evidence | established

// Beispiel: strukturell valide Kombination:
category = MODEL
modality = established   // Ein Modell kann gut etabliert sein

// Verbotene Konfusion (Protokollverletzung):
// Normative claims dürfen nicht modality = evidence tragen
// ohne explizite Begründung und category-Trennung.
```

Teil II — Vollständige Claimstruktur (v2)

Claim (Alexandria-protokollkonform, v2)	
claim_id	string (global unique, immutable)
subject	ref: Entity | Concept
predicate	ref: RelationType
object	ref: Entity | Concept | Claim
category	enum: EMPIRICAL | NORMATIVE | MODEL | SPECULATIVE ◄ *geändert*
qualifiers	map<string, any>
scope	map<string, any>
time_scope	map<string, any>
modality	enum: hypothesis | suggestion | evidence | established ◄ *geändert*
uncertainty	{ sigma, ci, n, type } // ersetzt confidence ◄ *geändert*
status	enum: UNVALIDATED | VALIDATED | SEALED | ◄ *geändert*
	FORMAL_ERROR | STABLE_AMBIGUITY | DEPRECATED | ◄ *geändert*

	partially_supported* | derived* (* DBA-Erweiterungen) ◄ *geändert*
evidence_refs[]	ref: Evidence[]
source_refs[]	ref: Work[]
assumptions[]	string[] // explizite Annahmen (Protokollpflicht) ◄ *geändert*
validation	{ validated: bool, decay: float, refs: [] } ◄ *geändert*
builder_origin	enum: alpha | beta | adjudicated // DBA-Erweiterung
created_at	timestamp (ISO 8601, monoton)
version	int
lineage[]	patch_id[] // Patch-Kette (Protokollpflicht) ◄ *geändert*

Markierte Felder (◄ geändert) sind entweder Korrekturen gegenüber v1 oder explizite Erweiterungen der Dual-Builder-Architektur (DBA). Nicht markierte Felder waren bereits protokollkompatibel.

Teil III — Neue und korrigierte Felder

assumptions[]

Protokollpflicht (Section VII.13.2, X.4). Jeder Claim muss seine Annahmen explizit listen. Implizite Annahmen sind ein formaler Audit-Fehler. Im Dual-Builder-Kontext werden Annahmen auch als Diff-Signal verwendet – Assumption-Divergenz zwischen Alpha und Beta ist ein eigenständiger Diff-Typ.

```
assumptions = [
  "Measurement_Calibrated_v1",
  "PopulationScope_KnowledgeWorkers",
  "TemporalScope_2018_2022"
]
```

uncertainty { sigma, ci, n, type }

Ersetzt confidence. Für probabilistische Claims sind alle vier Felder zu befüllen. Für deterministische Claims: type = deterministic, übrige Felder leer. Uncertainty wird nie gemittelt – auch nicht durch Adjudication. Divergierende Uncertainty-Werte aus Alpha und

Beta werden als confidence_divergence_major oder confidence_divergence_minor geloggt (Diff-Ontologie Anhang B).

```
uncertainty = {
  sigma:  0.11,
  ci:     [0.53, 0.75],
  n:      420,
  type:   "probabilistic"
}
```

validation { validated, decay, refs }

Entspricht dem Audit-Metadata-Feld aus dem Protokoll (Section VII.13.6). `decay` ist der λ-Parameter für die Stability-Berechnung (Section XI.4). Er muss explizit deklariert sein – Protokollpflicht. In einer produktiven Implementierung wird λ domänenspezifisch kalibriert; in der Reference Implementation ist ein flacher Wert akzeptabel.

```
validation = {
  validated:  true,
  decay:      0.01,       // λ-Parameter für Stability-Berechnung
  refs:       ["VAL_EMPIRICAL_REPLICATION"]
}
```

lineage[]

Protokollpflicht. Die geordnete Sequenz aller Patches, die diesen Claim verändert haben. Zusammen mit dem initialen Patch bildet dies die Epistemic Identity des Claims: ID(k) = Hash(initial_patch, lineage) – Section XI.2. Ohne Lineage ist der Claim im Protokoll nicht eindeutig identifizierbar.

```
lineage = ["patch_001", "patch_003", "patch_007"]
```

category (explizit, protokollkonform)

In v1 fehlend. Jeder Claim trägt eine explizite Kategorie nach Protokoll Section III.3. Kategorienmischung ist ein formaler Fehler (Audit Block I). Im Dual-Builder-Kontext ist entity_type_mismatch auch eine Kategoriendifferenz – der Diff-Typ aus Anhang B.1 deckt diesen Fall ab.

EMPIRICAL	Aussage über beobachtbare Fakten, Messungen, Daten
NORMATIVE	Soll-Aussage, Werturteil, moralisches oder politisches Postulat
MODEL	Vereinfachung, Abstraktion, theoretisches Rahmenwerk
SPECULATIVE	Annahme ohne gesicherte empirische Basis

Teil IV — Beispiel: vollständiger Claim (v2)

```
claim_id:         C-2048

subject:          RemoteWork
predicate:        CORRELATES_WITH
object:           Productivity

category:         EMPIRICAL               // neu in v2

qualifiers:
  sector:         software development
  measurement:    output per hour

scope:
  population:     knowledge workers
  region:         OECD countries

time_scope:
  start_year:     2018
  end_year:       2022

modality:         evidence                // orthogonal zu category

uncertainty:                              // ersetzt confidence in v2
  sigma:          0.11
  ci:             [0.53, 0.75]
  n:              420
  type:           probabilistic

status:           STABLE_AMBIGUITY        // protokollkonform (vorher:
disputed)

assumptions:
  - Measurement_Calibrated_v1             // neu in v2, Protokollpflicht
  - PopulationScope_KnowledgeWorkers
  - TemporalScope_2018_2022

evidence_refs:
  - evidence_443
  - evidence_889

source_refs:
  - work_1203
  - work_8891

validation:
  validated:      true
  decay:          0.01
  refs:           [VAL_EMPIRICAL_REPLICATION]

builder_origin:   adjudicated
lineage:          [patch_012, patch_019]  // neu in v2, Protokollpflicht
version:          2
```

created_at: 2026-03-10T18:42:00Z

Teil V — Claim-Lifecycle (protokollkonform)

Der Lifecycle orientiert sich an den Protokoll-Statusklassen (Section X.8). Der Übergang von VALIDATED zu SEALED erfordert Temporal Stability und Structural Integrity gemäß Anhang D.

UNVALIDATED	VALIDATED	SEALED	STABLE_AMB.	FORMAL_ERR.	DEPRECATED

UNVALIDATED	Builder hat Claim extrahiert; Audit noch nicht vollständig
VALIDATED	Alle 5 Audit-Blocks bestanden (Section X.3–X.7)
SEALED	Temporal Stability und Structural Integrity erfüllt; unveränderlich
STABLE_AMBIGUITY	Zwei konsistente, unvereinbare Pfade; Branch erforderlich
FORMAL_ERROR	Strukturverletzung; Claim bleibt erhalten als FORMAL_ERROR-Record
DEPRECATED	Durch neuere Version oder Widerlegung ersetzt; bleibt in Lineage
partially_supported*	DBA-Erweiterung: konditionale Stützung; keine direkte Protokolläquivalenz
derived*	DBA-Erweiterung: aus anderen Claims abgeleitet; keine direkte Protokolläquivalenz

** Diese beiden Statuswerte sind begründete Erweiterungen der Dual-Builder-Architektur.* Sie sind im Protokoll nicht definiert und müssen in einer protokollkonformen Implementierung explizit als Erweiterungen deklariert werden.

Ende von Anhang E (v2)

Struktur dieses Anhangs

F.1 Grundprinzipien der Relationsontologie

F.2 Kausalitätsskala — Zwei-Ebenen-Modell (Kernbereich)

F.3 Claim-zu-Claim-Relationen (CONTRADICTS, REFINES, DERIVED_FROM, …)

F.4 Evidenz-Relationen (HAS_EVIDENCE, SUPPORTED_BY, MENTIONED_IN)

F.5 Ontologische und wissenschaftliche Relationen (Kurzreferenz)

F.6 Adjudication-Prioritätstabelle (vollständig)

F.7 Verbindung zu Anhang B (Diff-Typen) und Anhang C (Regelklassen)

F.1 — Grundprinzipien der Relationsontologie

Die Relationsontologie des Alexandria-Protokolls folgt vier bindenden Prinzipien, die alle Relationsdefinitionen und Inferenzregeln durchziehen.

F.1.1 Inferenzschutz als Designprinzip

Jede Relation definiert nicht nur, was sie bedeutet, sondern explizit, was aus ihr nicht geschlossen werden darf. Die Runtime darf nur solche Inferenzen ziehen, die im Anhang F positiv gelistet sind. Alle anderen Schlüsse sind verboten. Fehlende Erlaubnis gilt als Verbot.

F.1.2 Keine implizite Implikation zwischen Stufen

Obwohl die Kausalitätsskala eine semantische Ordnung darstellt (jede stärkere Relation macht eine stärkere epistemische Behauptung), impliziert eine höhere Stufe nicht automatisch alle niedrigeren. Wer CAUSES behauptet, behauptet nicht damit auch CORRELATES_WITH — weil die Annahmemengen verschieden sind und der Claim explizit mit seiner Relation versiegelt wurde.

F.1.3 Relationstyp ist Bestandteil der epistemischen Identität

Der Relationstyp eines Claims ist Teil seines Inhalts. Eine Änderung des Relationstyps ist eine MODIFY-Operation mit neuer Lineage, nicht eine interne Umbenennung. `CORRELATES_WITH` und `CAUSES` sind epistemisch verschiedene Claims, auch wenn Subject und Object identisch sind.

F.1.4 Kategorienpurität bindet Relationsauswahl

Die Wahl des Relationstyps ist an die Kategorie des Claims gebunden. Ein NORMATIVE-Claim darf keine kausale Relation tragen, die eine empirische Behauptung impliziert. Ein SPECULATIVE-Claim darf CAUSES nicht tragen — höchstens MIGHT_CAUSE als Erweiterung oder CORRELATES_WITH mit entsprechender Modality.

F.2 — Kausalitätsskala — Zwei-Ebenen-Modell

Die Kausalitätsskala ist das zentrale epistemische Unterscheidungsproblem des Alexandria-Graphen. Sie beschreibt, wie stark ein Claim die Verbindung zwischen Subject und Object behauptet.

F.2.1 Semantische Ordnung (Ebene 1)

Die acht Relationsstufen sind in aufsteigender epistemischer Stärke geordnet. Jede Stufe erfordert stärkere Evidenzbasis und explizitere Annahmen als die vorherige.

MENTIONS	RELATES_TO	CORRELATES_WITH	PARTIALLY_SUPPORTS	SUPPORTS	STRONGLY_SUPPORTS	CONTRIBUTES_TO	CAUSES
Erwähnung	*Assoziation*	*Korrelation*	*part. Stützung*	*Stützung*	*starke Stützung*	*Beitrag*	*Kausalität*

Wichtig: Diese Ordnung ist semantisch, nicht hierarchisch im Sinne logischer Implikation. Siehe F.1.2.

F.2.2 Definitionen und Inferenzregeln (Ebene 2)

MENTIONS

MENTIONS
Bedeutung: Subject wird im Kontext von Object erwähnt. Keine inhaltliche Beziehungsbehauptung.
Evidenzbasis: Reine Kookkurrenz im Text genügt. Keine Analyse erforderlich.
Typische Quelle: Abstracts, Literaturlisten, Glossareinträge.
Abgrenzung: Sobald eine inhaltliche Verbindung behauptet wird, ist RELATES_TO zu wählen.

✓ Erlaubte Inferenzen	✗ Verbotene Inferenzen
Subject und Object kommen im selben epistemischen Kontext vor.	Kein Schluss auf inhaltliche Verbindung.
MENTIONS ist konsistent mit allen stärkeren Relationstypen.	Kein Schluss auf Korrelation oder Kausalität.
	MENTIONS darf nicht als Evidenz für SUPPORTS verwendet werden.

RELATES_TO

RELATES_TO
Bedeutung: Subject und Object stehen in einer inhaltlichen Verbindung, deren Natur nicht näher

bestimmbar ist.
Evidenzbasis: Inhaltliche Analyse zeigt Verbindung, aber Richtung oder Typ unklar.
Typische Quelle: Reviews, Einleitungsabschnitte, exploratorische Texte.
Abgrenzung: Wenn Richtung oder Typ bestimmbar: spezifischere Relation verwenden. RELATES_TO ist Auffangkategorie.

✓ Erlaubte Inferenzen	✗ Verbotene Inferenzen
Inhaltliche Verbindung existiert.	Kein Schluss auf Richtung der Verbindung.
Builder-Diff auf RELATES_TO ist schwaches Signal — Adjudication kann vereinheitlichen.	Kein Schluss auf Korrelation, Kausalität oder Unterstützung.
	Nicht als Evidenz für SUPPORTS oder CAUSES verwenden.

CORRELATES_WITH

CORRELATES_WITH
Bedeutung: Subject und Object ko-variieren systematisch in beobachtbaren Daten. Keine Aussage über Richtung oder Kausalität.
Evidenzbasis: Statistische Analyse erforderlich. Uncertainty-Tuple (sigma, ci, n) Pflicht.
Symmetrie: Symmetrisch: A CORRELATES_WITH B ⇔ B CORRELATES_WITH A.
Abgrenzung: Wenn Richtungsasymmetrie beobachtet: PARTIALLY_SUPPORTS oder SUPPORTS. Wenn kausaler Mechanismus bekannt: CONTRIBUTES_TO oder CAUSES.

✓ Erlaubte Inferenzen	✗ Verbotene Inferenzen
Systematische Ko-variation in den Daten.	Kein Schluss auf Kausalität.
Uncertainty-Tuple ist auswertbar.	Kein Schluss auf Richtung.
Basis für weitere Untersuchung von Richtung und Kausalität.	Kein Schluss auf mechanistische Erklärung.

	CORRELATES_WITH + hohe Korrelation ≠ CAUSES.

PARTIALLY_SUPPORTS

PARTIALLY_SUPPORTS
Bedeutung: Subject liefert unter bestimmten Bedingungen Evidenz für Object, aber nicht unter allen. Bedingte Unterstützung.
Evidenzbasis: Empirische Analyse mit explizitem scope{} und qualifiers{}. Uncertainty-Tuple Pflicht.
Richtung: Gerichtet: Subject → Object. Nicht symmetrisch.
Abgrenzung: Wenn Unterstützung bedingungslos gilt: SUPPORTS. Wenn nur Korrelation: CORRELATES_WITH.

✓ Erlaubte Inferenzen	✗ Verbotene Inferenzen
Unterstützungsbeziehung gilt innerhalb des deklarierten scope{}.	Kein Schluss auf SUPPORTS ohne scope-Analyse.
Kann als schwache Prämisse in Argumentationsketten verwendet werden (mit explizitem scope-Verweis).	Kein Schluss auf CAUSES.
	Ausserhalb des deklarierten scope{} keine Inferenz zulässig.

SUPPORTS

SUPPORTS
Bedeutung: Subject liefert substanzielle, robuste Evidenz für Object über verschiedene Kontexte hinweg.
Evidenzbasis: Multiple unabhängige Studien oder Replikationen. Uncertainty-Tuple mit kleinem sigma.
Richtung: Gerichtet. Nicht symmetrisch.
Abgrenzung: Wenn Evidenz noch konditionaler: PARTIALLY_SUPPORTS. Wenn mechanistische Erklärung bekannt: CONTRIBUTES_TO oder CAUSES.

✓ Erlaubte Inferenzen	✗ Verbotene Inferenzen
Subject ist relevante Evidenz für Object.	Kein Schluss auf CAUSES.
Runtime darf SUPPORTS als Prämisse in Argumentationsketten nutzen.	Kein Schluss auf STRONGLY_SUPPORTS ohne zusätzliche Evidenz.
Mehrere unabhängige SUPPORTS-Relationen erhöhen epistemische Stabilität von Object.	SUPPORTS ≠ bewiesen.

STRONGLY_SUPPORTS

STRONGLY_SUPPORTS
Bedeutung: Subject liefert starke, breit replizierte Evidenz für Object. Höchste Unterstützungsstufe ohne Kausalbehauptung.
Evidenzbasis: Meta-Analysen, systematische Reviews, sehr kleines sigma. Mehrfache unabhängige Replikationen.
Richtung: Gerichtet.
Abgrenzung: Wenn kausaler Mechanismus bekannt und geprüft: CAUSES. Ohne Mechanismus: bei STRONGLY_SUPPORTS bleiben.

✓ Erlaubte Inferenzen	✗ Verbotene Inferenzen
Höchste verfügbare Unterstützung ohne Kausalbehauptung.	Kein automatischer Schluss auf CAUSES.
Runtime darf als starke Prämisse verwenden.	STRONGLY_SUPPORTS impliziert keinen bekannten Mechanismus.
Basis für SEALED-Status des Object-Claims.	Hohe Korrelation allein genügt nicht für STRONGLY_SUPPORTS.

CONTRIBUTES_TO

CONTRIBUTES_TO
Bedeutung: Subject ist ein kausaler Faktor für Object, aber nicht der alleinige. Partieller Kausalanspruch mit bekanntem Beitragsmechanismus.

Evidenzbasis: Experimentelle oder quasi-experimentelle Evidenz für Kausalität. Mechanismus mümss zumindest hypothetisch beschreibbar sein.	
Richtung: Gerichtet. Asymmetrisch.	
Abgrenzung: Wenn Faktor notwendig und hinreichend: CAUSES. Wenn Mechanismus nicht bekannt: SUPPORTS oder STRONGLY_SUPPORTS.	

✓ Erlaubte Inferenzen	✗ Verbotene Inferenzen
Subject ist kausaler Beitragsfaktor für Object.	Kein Schluss auf vollständige Kausalität (CAUSES).
Runtime darf kausale Teilbeziehung annehmen.	Entfernen von Subject eliminiert Object nicht notwendigerweise.
Relevant für Interventionsargumente (aber nicht allein hinreichend).	Kein Schluss auf Ausschluss anderer Kausalfaktoren.

CAUSES

CAUSES
Bedeutung: Subject ist notwendige und/oder hinreichende Ursache für Object unter deklarierten Bedingungen. Stärkste epistemische Behauptung.
Evidenzbasis: Randomisierte kontrollierte Studien, natürliche Experimente oder formale kausale Modelle (z. B. DAGs mit d-Separation). Uncertainty-Tuple mit explizitem CI.
Annahmen: assumptions[] muss Kontrollbedingungen, Confounding-Ausschluss und Mechanismusbeschreibung enthalten.
Abgrenzung: Wenn nur Beitragsfaktor: CONTRIBUTES_TO. Wenn nur Korrelation: CORRELATES_WITH. Ohne formale Kausalanalyse ist CAUSES verboten.

✓ Erlaubte Inferenzen	✗ Verbotene Inferenzen
Runtime darf Interventionsargumente ableiten (innerhalb des deklarierten scope{}).	Kein Schluss auf universelle Kausalität ausserhalb des scope{}.
Basis für kausale Erklärungen in Argumentationsketten.	Kein Schluss auf Umkehrbarkeit (CAUSES ist nicht symmetrisch).
Erlaubt kontrafaktische Inferenz: Ohne Subject wäre Object (unter gegebenen Bedingungen)	Kein Schluss auf Ausschluss aller anderen Ursachen.

anders.	
	CAUSES ohne deklarierte Annahmen und CI = FORMAL_ERROR.

F.3 — Claim-zu-Claim-Relationen

Diese Relationen verbinden nicht Subject und Object eines Claims, sondern zwei Claims miteinander. Sie sind das zentrale Werkzeug für epistemischen Dissens, Präzisierung und Derivation.

Relation	Bedeutung + Bedingungen	Erlaubte Inferenz der Runtime
CONTRADICTS	Claim A und Claim B sind unter denselben Annahmen nicht gleichzeitig wahr. Bedingung: gleicher scope{} und gleiche assumptions[]. Sonst: assumption_mismatch statt CONTRADICTS.	Dissens ist dokumentiert. Branch erforderlich (Section VII.13.4). Status beider Claims: STABLE_AMBIGUITY.
PARTIALLY_CONTRADICTS	A und B sind unter einem Teil der Annahmen unvereinbar. Teilkonflikt mit identifizierbarer Schnittmenge.	Partieller Dissens sichtbar. Adjudication prüft assumption-Trennung (Regel C.9).
REFINES	Claim B präzisiert Claim A ohne ihn zu widerlegen. B ist detaillierter, enger im scope oder mit kleinerer uncertainty. Bedingung: B ist mit A logisch kompatibel.	Runtime darf B als informativere Version von A verwenden. A erhält DERIVED_FROM-Markierung.
GENERALIZES	Claim B verallgemeinert Claim A. B hat grösseren scope{} oder schwächere modality. Inverse von REFINES.	Runtime darf B als übergeordneten Rahmen verwenden. Kein Schluss: B ist daher besser belegter.
DERIVED_FROM	Claim B wurde logisch oder inferenziell aus Claim A abgeleitet. A ist Prämisse, B ist Konklusion.	Uncertainty-Propagation Pflicht: U(B) ≥ f(U(A)) gemäss Section VII.6. Validität von B hängt von A ab.
REPLACES	Claim B tritt an die Stelle von Claim A. A wird DEPRECATED, bleibt aber in der Lineage sichtbar.	A: status = DEPRECATED. B trägt A in lineage[]. Kein retroaktives Löschen.
EXTENDS	Claim B erweitert den Geltungsbereich oder die	Beide Claims bleiben aktiv.

	Dimensionen von Claim A, ohne ihn zu ersetzen.	Kein Implikationsschluss: B gilt nicht automatisch dort, wo A gilt.

Protokollbindung für CONTRADICTS und DERIVED_FROM

CONTRADICTS: Nur zulässig bei identischem scope{} und überlappenden assumptions[]. Anderenfalls assumption_mismatch (Diff B.8).

DERIVED_FROM: Uncertainty-Propagation ist Protokollpflicht (Section VII.6). Fehlt sie, ist der Claim UNVALIDATED.

F.4 — Evidenz-Relationen

Evidenz-Relationen verbinden Claims mit konkreten Belegen. Sie sind strukturell einfacher als die Kausalitätsskala, aber epistemisch unverzichtbar: ohne sie ist kein Claim auditierbar.

Relation	Bedeutung	Inferenzregel
`HAS_EVIDENCE`	Claim ist durch eine konkrete Evidence-Instanz belegt. Evidence-Knoten trägt Quelle, Textstelle und eigenes uncertainty-Tuple.	Claim ist prinzipiell auditierbar. Kein Schluss auf Qualität der Evidenz.
`SUPPORTED_BY`	Claim wird durch ein Work (Publikation, Datensatz) gestützt. Allgemeiner als HAS_EVIDENCE.	Work ist relevante Quelle. Kein Schluss auf Reproduzierbarkeit.
`MENTIONED_IN`	Claim oder Entität wird in einem Work erwähnt, ohne dass eine Unterstützungsbeziehung besteht.	Kein Schluss auf Unterstützung. Nur: Kookkurrenz im Quelltext.
`CONTRADICTED_BY`	Ein Work liefert Evidenz gegen den Claim. Formalere Version als CONTRADICTS (Work vs. Claim statt Claim vs. Claim).	Status des Claims: mindestens STABLE_AMBIGUITY. Adjudication-Pflicht.
`REPLICATED_BY`	Ein unabhängiges Work reproduziert die dem Claim zugrundeliegenden Befunde.	Erhöht stability gemäss XI.4. Kein Schluss auf Unfehlbarkeit.

F.5 — Ontologische und wissenschaftliche Relationen (Kurzreferenz)

Diese Relationsgruppen sind im Protokoll strukturell definiert (Anhang A). Anhang F ergänzt die Inferenzregeln.

Relation	Bedeutung	Inferenzregel
`INSTANCE_OF`	Entität ist Instanz eines Konzepts. Transitiv über SUBCLASS_OF.	Runtime: Konzepteigenschaften gelten für Instanz. Keine umgekehrte Instanziierung.
`SUBCLASS_OF`	Konzept ist Unterklasse. Transitiv.	Eigenschaften der Oberklasse vererbt. Nicht umgekehrt.
`PART_OF`	Strukturelle Teilbeziehung. Nicht transitiv ohne explizite Deklaration.	Kein Schluss auf Eigenschaftsvererbung.
`AUTHORED_BY`	Work wurde von Author verfasst.	Kein Schluss auf institutionelle Zugehörigkeit zum Zeitpunkt der Publikation.
`CITES`	Work zitiert ein anderes Work.	Kein Schluss auf inhaltliche Unterstützung (nur strukturelle Verknüpfung).

F.6 — Adjudication-Prioritätstabelle

Wenn Alpha und Beta unterschiedliche Relationstypen für denselben Claim wählen, entscheidet Regel C.7 (epistemisch defensivere Relation gewinnt). Diese Tabelle macht die Prioritätsordnung vollständig explizit.

Prio	Relation	Zulässige Inferenz	Adjudication
1	`MENTIONS`	Kookkurrenz im Quellkontext vorhanden.	`C.7`
2	`RELATES_TO`	Inhaltliche Verbindung vorhanden (Typ unbestimmt).	`C.7`
3	`CORRELATES_WITH`	Systematische Ko-variation in Daten belegt.	`C.7`
4	`PARTIALLY_SUPPORTS`	Konditionale Unterstützung im deklarierten scope{}.	`C.7`

5	SUPPORTS	Robuste Unterstützung über Kontexte hinweg.	C.7
6	STRONGLY_SUPPORTS	Breite Replikation, Meta-Analyse.	C.7
7	CONTRIBUTES_TO	Partieller kausaler Beitrag mit Mechanismus.	C.7
8	CAUSES	Vollständige Kausalität mit formaler Analyse.	C.7

Adjudication-Logik bei Dissens (Regel C.7)

Bei Dissens zwischen Alpha und Beta über den Relationstyp gewinnt die Relation mit der niedrigeren Prioritätsstufe (defensiver).

Ausnahme: Wenn der defensivere Typ einen anderen epistemischen Typ (z. B. CORRELATES_WITH vs. CONTRIBUTES_TO) impliziert,

ist ein causality_mismatch-Diff (B.4) zu loggen und Adjudication-Regel C.3 (Konfliktfall) anzuwenden.

Kategorienkonflikte (EMPIRICAL vs. MODEL) gehen vor und werden nach Regel C.8 behandelt.

F.7 — Verbindung zu Anhang B und Anhang C

Anhang F, B und C bilden ein geschlossenes System: F definiert die Relationen, B typisiert die möglichen Diffs, C bestimmt das Adjudication-Verfahren.

Relation	Bedeutung	Inferenzregel
relation_mismatch (B.4)	Zwei Builder wählen verschiedene Relationstypen auf der Kausalitätsskala.	Adjudication: C.7 (Prioritätstabelle F.6)
causality_mismatch (B.4)	Ein Builder liest Kausalität, der andere nur Korrelation.	Adjudication: C.3 + C.7 kombiniert
claim_modality_mismatch (B.3)	Unterschiedliche epistemische Stärke: modality-Feld, nicht Relationstyp.	Adjudication: C.1
assumption_mismatch (B.8)	Unterschiedliche assumptions[] führen zu CONTRADICTS statt echtem Widerspruch.	Adjudication: C.9 — assumption-Trennung prüfen
category_mismatch (B.9)	EMPIRICAL vs. MODEL führt zu unterschiedlicher Relationsauswahl.	Adjudication: C.8 — geht vor C.7

`DERIVED_FROM (F.3)`	Uncertainty-Propagation ist Pflicht (Section VII.6).	Fehlt sie: claim = UNVALIDATED
`CONTRADICTS (F.3)`	Nur zulässig bei identischem scope{} und überlappenden assumptions[].	Sonst: assumption_mismatch (B.8)

Invariante: Jeder Diff-Typ in Anhang B hat mindestens eine Regel in Anhang C. Jede Regel in Anhang C setzt mindestens eine Relationsdefinition in Anhang F voraus. Das System ist geschlossen.

Ende von Anhang F — Relationsontologie

TECHNICAL ANNEX G
Graph Maturity Metrics

Alexandria Protocol v2.1 Extension

G.1 Motivation und Scope

Das Protokoll spezifiziert den Lebenszyklus individueller Claims — von **UNVALIDATED** über **VALIDATED** zu **SEALED**. Was das Kernprotokoll nicht adressiert: die Bereitschaft des *Graphen als Ganzes*. Ein Graph mit ungelösten HIGH-Diffs, unvollständiger Source-Traceability oder schwerem Claim-Decay ist epistemisch nicht bereit — unabhängig vom Status einzelner Claims.

Annex G führt fünf **Graph Maturity Metrics** (M1–M5) und einen Composite-Score **Φ** (Phi) ein, der die epistemische Bereitschaft des gesamten Alexandria-Graphen misst.

- **Berechenbar —** aus Graphzustand ohne externe Annotation
- **Monoton —** verbesserte Qualität kann Φ nicht verringern
- **Semantisch verankert —** jede Metrik mappt auf eine Protokollanforderung
- **Nicht manipulierbar —** keine einzelne Metrik trivial maximierbar

G.2 Metriken M1–M5

Alle Metriken normiert auf **[0.0, 1.0]**. Referenzimplementierung: `alexandria_core/maturity.py`.

G.2.1 M1 — GraphDensity

Strukturelle Abdeckung: Statusverteilung, Kategorienbreite, Prädikatendiversität.

```
M1 = 0.4 · status_score + 0.3 · category_score + 0.3 · predicate_diversity
```

Komponente	Definition
`status_score`	Anteil Claims außerhalb UNVALIDATED / FORMAL_ERROR

Komponente	Definition
`category_score`	Vorhandene Kategorien / 4
`predicate_diversity`	Genutzte Prädikate / 8 (volle Kausalskala)

G.2.2 M2 — ClaimStability

Erweitert $S_k(t)$ aus **Section XI.4** auf den gesamten Graphen:

```
S_k(t) = e^(-λ_k · Δt)     [Δt in Tagen seit created_at]

M2 = (1/N) · Σ S_k(t)
```

Claims ohne λ-Parameter: Fixwert 0.5. Höchste Gewichtung (w_2 = 0.25), da temporale Stabilität der primäre Qualitätsindikator ist.

G.2.3 M3 — DiffResolutionRate

Adjudication-Vollständigkeit. Schweregrad-gewichtete Auswertung:

```
M3 = Σ(resolved_weight) / Σ(total_weight)
```

- `HIGH   × 3` (ungelöst → blockiert Sealing, Kriterium D.1)
- `MEDIUM × 2`
- `LOW    × 1`

M3 = 1.0 wenn keine Diffs existieren (trivial konsistenter Graph).

G.2.4 M4 — EvidenceCoverage

Operationalisiert Seal-Kriterium D.2 und Section VII.4 direkt:

```
M4 = 0.40 · source_coverage + 0.35 · uncertainty_disclosure + 0.25 · assumption_coverage
```

Komponente	Definition
`source_coverage`	Anteil Claims mit ≥1 source_ref oder evidence_ref
`uncertainty_disclosure`	Anteil EMPIRICAL-Claims mit Uncertainty-Tuple {sigma, ci, n}
`assumption_coverage`	Anteil Claims mit nicht-leerer assumptions[]

G.2.5 M5 — TemporalPersistence

Misst, wie lange Claims ohne Revision persistieren. Logistische Sättigungskurve über Zeit seit letztem MODIFY-Patch:

```
score_k = 1 − e^(-Δt_k / 10)     [Δt_k in Tagen]

M5 = (1/N) · Σ score_k
```

Ohne Patch-Chain: Anteil Claims älter als 24h (konservativer Fallback).

G.3 Composite Score Φ

$$\Phi = w_1 \cdot M1 + w_2 \cdot M2 + w_3 \cdot M3 + w_4 \cdot M4 + w_5 \cdot M5$$

Metrik	Gewicht wi	Bereich	Blockiert Seal wenn
M1 — GraphDensity	0.15	[0,1]	—
M2 — ClaimStability	0.25	[0,1]	< 0.30
M3 — DiffResolutionRate	0.25	[0,1]	< 0.50
M4 — EvidenceCoverage	0.20	[0,1]	< 0.40
M5 — TemporalPersistence	0.15	[0,1]	—

Gewichte konfigurierbar via `MaturityCalculator(weights={...})`. Bedingung: Σ wi = 1.0.

G.4 Maturity-Schwellwerte und erlaubte Aktionen

Φ	Level	Bedeutung	Erlaubte Aktionen
≥ 0.85	**MATURE**	Hohe epistemische Stabilität, alle Diffs gelöst	SEALED-Status, Produktiveinsatz

Φ	Level	Bedeutung	Erlaubte Aktionen
≥ 0.65	**FUNCTIONAL**	Nutzbarer Graph, geringe Lücken	VALIDATED-Status, Exploration
≥ 0.40	**DEVELOPING**	Signifikante Lücken, aktiver Aufbau	Nur interne Entwicklung
< 0.40	**IMMATURE**	Nicht bereit	Kein externer Zugriff

Unabhängig von Φ blockieren folgende Zustände den Seal: FORMAL_ERROR-Claims (D.3), offene HIGH-Diffs (D.1), Chain-Integritätsfehler (D.4).

G.5 Audit Trail und Trend-Tracking

Jeder **MaturityReport** ist unveränderlich nach Erzeugung. Er trägt `timestamp` und `graph_id` und kann als `:MaturitySnapshot`-Node im Neo4j-Graph gespeichert werden.

Die **MaturityTrend**-Klasse akkumuliert Reports über Zeit und erkennt:

- **Verbesserung:** $\Delta\Phi > 0$ zwischen aufeinanderfolgenden Snapshots
- **Regression:** $\Delta\Phi < -0.02$ — löst Warnung aus
- **Stagnation:** Φ stabil aber unter FUNCTIONAL seit >7 Tagen — strukturelle Blockade

Regression während aktiver DBA-Verarbeitung signalisiert oft ein systematisches Extraktionsproblem in einem Builder — quantifizierbar durch **BuilderBiasAnalyzer** (Annex B / diff.py).

G.6 Protokollbezug

Metrik	Protokollabschnitt	Operationalisierte Anforderung
M1	III.3, X.8	Kategorienabdeckung und Statusverteilung
M2	XI.4	$S_k(t)$ individuell → Graph-Mittelwert
M3	V-A.6, Annex B/C	Adjudication-Vollständigkeit über alle DiffNodes

Metrik	Protokollabschnitt	Operationalisierte Anforderung
M4	VII.4, D.2	Source-Traceability und Uncertainty-Disclosure
M5	XI.4, VII.13.7	Temporale Persistenz via Patch-Chain-Historie
Φ	Annex D (D.5)	Composite-Bereitschaft für SEALED-Übergang

G.7 Implementierungshinweise

Referenzimplementierung: `alexandria_core/maturity.py`. Alle Metriken werden aus In-Memory-Claim-Listen und einer PatchChain-Instanz berechnet. Keine Datenbankabfragen erforderlich.

G.7.1 Assessment-Frequenz

- Nach jeder vollständig verarbeiteten Quellquelle (DBA-Pipeline)
- Vor jedem Seal-Versuch
- Täglich zur Decay-Überwachung bestehender Claims
- Nach jeder Bulk-DEPRECATE-Operation

G.7.2 Gewichtskalibrierung

Anpassungen via `MaturityCalculator(weights={...})`, Bedingung Σ wi = 1.0. Änderungen müssen als `:MaturityConfig`-Node im Graph dokumentiert werden.

G.7.3 Minimale Graphgröße

Aussagekräftig ab **N ≥ 10 Claims**. Bei N < 10 dominiert Stichprobenrauschen M1 und M3. MaturityCalculator gibt ab N = 1 einen Report, ergänzt aber bei N < 10 einen Hinweis.

Ende von Annex G — Graph Maturity Metrics

Technical Annex H — v2.2 Structural Changes

Sprint 1 – Sprint 2 – Sprint 3 · Hanns-Steffen Rentschler · 2026

This annex documents all normative changes introduced in v2.2 relative to v2.1. Changes are organized by sprint. Each change carries an explicit norm level: [SHALL], [DBA], [HEURISTIC], or [ADVISORY].

H.1 Sprint 1 — Four Structural Fixes

H.1.1 Normebenen-Trennung [SHALL]

v2.2 introduces an explicit three-level norm separation. Every statement in the protocol, annexes, and implementation must be unambiguously assigned to one of: [SHALL] Protocol invariant (never deviate — violation is FORMAL_ERROR); [DBA] Dual-Builder Architecture extension (protocol-compliant, explicitly declared); [HEURISTIC] Reference implementation default (replaceable, not normative). A fourth tag [ADVISORY] marks recommendations without formal status. Mixing levels is a formal error.

H.1.2 Catch-all Adjudication: No Silent Winner [SHALL]

In v2.1, rule C.3 produced a silent default winner (Alpha) when no specific rule matched. This violated the protocol spirit of keeping dissent visible. In v2.2: no diff-type may be silently resolved. An unknown diff-type MUST surface as UNRESOLVED_PENDING_RULE with winning_id = None. C.3 is now an escalation rule, not a resolution rule. Allowed fallbacks: STABLE_AMBIGUITY (structurally unresolvable), FORMAL_ERROR (category violation), BRANCH_REQUIRED (explicit branch needed), UNRESOLVED_PENDING_RULE (no match in C.1–C.9). Forbidden: any silent default-win without an explicit rule.

H.1.3 BranchNode as First-Class Object [DBA]

Branching is the normal case in Alexandria, not an exception. In v2.1, a branch was an implicit status-pair (claim_alpha, claim_beta). In v2.2, BranchNode is a formal DBA object with full schema: branch_id (UUID), parent_branch_id, trigger_diff_ids, branch_reason, branch_scope, claim_alpha_id, claim_beta_id, merge_policy (NEVER | ON_NEW_EVIDENCE | ON_SCOPE_CHANGE | ON_RULE_EXTENSION | HUMAN_REVIEW_ONLY), status (OPEN | MERGED | DEPRECATED | ARCHIVED), created_at, deprecated_at, deprecation_reason. Silent disappearance of branches is forbidden: every branch must have a lifecycle up to DEPRECATED or ARCHIVED. db.py exposes upsert_branch() for Neo4j persistence.

H.1.4 Seal Decoupled from Maturity: D.5 Advisory Only [SHALL]

Seal is a formal admissibility decision based on structural invariants. Maturity (Φ) is an operational readiness metric. A heuristic (weighted sum of proxies) must not block a formally correct seal. In v2.2: D.5 is ADVISORY ONLY — always passed = True, never blocks sealing. The Φ-value is logged in SealRecord for operational use. Hard seal criteria are D.1 (no open HIGH diffs), D.2 (source traceability), D.3 (no FORMAL_ERROR), D.4 (patch chain integrity), D.6 (non-empty assumptions). D.5 is not in this set.

H.2 Sprint 2 — Four Formalizations

H.2.1 Claim–Patch–Lineage Doctrine [SHALL]

The epistemic primary unit is not the Claim alone. It is: Claim + Lineage + Patch History. A Claim is a readable epistemic snapshot. A Patch is the only valid state transition ([SHALL]: no change without patch). Lineage is the ordered list of Patch IDs enabling full reconstruction. Reconstructability only arises from all three together. An audit of a Claim without its Lineage is incomplete.

EpistemicIdentity.is_complete(claim, patch_chain) checks: uncertainty present when required, assumptions[] non-empty, all lineage Patch IDs present in the Chain.

H.2.2 Relations Admissibility Matrix [DBA]

Sprint 2 adds a machine-checkable enforcement layer to the Relations Ontology (Annex F): RelationsMatrix (alexandria_core/relations.py). Without this matrix, builders can semantically drift: NORMATIVE/CAUSES is protocol-violating, but without enforcement it passes unnoticed. The matrix defines: which Category/Predicate combinations are allowed (violations → FORMAL_ERROR), which require an uncertainty tuple, and which trigger a branch on Alpha/Beta conflict. AuditGate Block I uses RelationsMatrix.validate_claim(). Adjudicator C.8 uses RelationsMatrix.requires_branch_on_conflict().

H.2.3 Three-Level Audit [DBA]

The existing AuditGate (5-block system, Section X) mixed Patch-, Claim-, and Graph-level checks. Sprint 2 separates these three levels explicitly as a complementary interface. Level 1 — Patch (PatchAuditResult) [SHALL]: timestamp monotonicity, parent_patch_id, mandatory fields, assumptions[]. Level 2 — Claim (ClaimAuditResult) [SHALL]: Category purity, assumptions, RelationsMatrix check, uncertainty, source references. Level 3 — Graph (GraphAuditResult) [DBA]: orphan detection, source traceability, branch coherence (OPEN branches older than threshold), referential integrity. The existing AuditGate is retained. ThreeLevelAudit is additive. Pipeline.py runs both in parallel.

H.2.4 Uncertainty Enforcement Rule [SHALL]

The v2.1 trigger (evidence/established → uncertainty required) was too broad, causing false positives on structural claims. The v2.2 rule: uncertainty is mandatory if and only if category = EMPIRICAL AND modality ∈ {evidence, established} AND predicate ∈ {CORRELATES_WITH, PARTIALLY_SUPPORTS, SUPPORTS, STRONGLY_SUPPORTS, CONTRIBUTES_TO, CAUSES}. These six predicates assert probabilistic empirical variance. For all other predicates, uncertainty is optional. The rule is enforced at three points: ClaimNode.validate() returns an error list, PatchEmitter.add() raises ValueError on validation failure, and AuditGate Block V checks EpistemicIdentity.uncertainty_required().

H.3 Sprint 3 — Four Refinements

H.3.1 Ontology Mapping as Uncertain Alignment [DBA]

OpenCyc (and any external ontology) provides candidates, not truth. Mapping a text to an ontology node is an alignment step with an explicit confidence level, not a safe lookup. Anti-pattern: treating concept mapping silently as ground truth. Sprint 3 introduces MappingConfidence (MAPPED, CANDIDATE, LOW_CONFIDENCE, MULTIPLE_CANDIDATES, UNMAPPED, EXCLUDED) and ConceptMappingResult, which carries confidence, confidence_score, matched_concept, candidates (top-5), is_usable, and requires_review. [DBA] Concepts mapped via UNMAPPED or LOW_CONFIDENCE may not enter the Alexandria graph without explicit human approval. is_usable = False → Concept remains in a staging queue until manually confirmed or rejected.

H.3.2 DiffNode Bias Metadata [DBA]

Sprint 3 adds three optional fields to DiffNode for systematic bias detection: adjudication_rule (str | None — which rule adjudicated this diff, e.g. ‘C.7’); winning_builder (‘alpha’ | ‘beta’ | None for ambiguous/unresolved); bias_tag (semantic label of the error pattern, e.g. ‘causal_overreach’, ‘scope_narrowing’, ‘hedge_missing’, ‘normative_drift’). BuilderBiasAnalyzer aggregates over all DiffNodes to identify recurring patterns. Bias Index (Builder X, Tag T) = count(winning_builder ≠ X AND bias_tag = T) / total_diffs_with_tag(T). Value > 0.7 indicates systematic bias. All three fields are [DBA] optional and do not change DiffNode semantics.

H.3.3 Uncertainty Enforcement: Full Chain [SHALL]

Sprint 2 defined the uncertainty rule conceptually. Sprint 3 enforces it throughout the full stack: ClaimNode.validate() → PatchEmitter.add() → AuditGate Block V → ThreeLevelAudit.audit_claim(). Claims that fail validate() cannot enter the chain via PatchEmitter. This is the enforcement mechanism, not a bug. Example: EMPIRICAL/established/CORRELATES_WITH without uncertainty → validate() error + Block V FAIL. NORMATIVE/SUPPORTS → uncertainty not applicable, no check. MODEL/CAUSES → uncertainty not required (model causality ≠ empirical causality).

H.3.4 Evaluation Framework [HEURISTIC]

A protocol implementation without an evaluation framework is incomplete. Sprint 3 defines six measurement levels: Claim Extraction (Precision/Recall vs. gold-annotated documents), Category Correctness (% correctly classified, target > 85%), Predicate Correctness (% correct, annotated against Causal Scale, Annex F.2), Uncertainty Calibration (Brier Score / ECE, declared CI vs. empirical coverage), Diff Resolution Rate (% resolved diffs per batch, metric M3), Mapping Quality (Accuracy / MRR, MappingConfidence vs. manual gold standard). The benchmark plan (5 phases) and all thresholds are [HEURISTIC] — replaceable. The protocol does not prescribe specific values.

H.4 Complete Changelog v2.1 → v2.2

Sprint 1

Sprint 1

Komponente	Alt (v2.1)	Neu (v2.2)
Normebenen	nicht explizit getrennt	Drei-Ebenen-Tabelle: SHALL / DBA / HEURISTIC
C.3 Catch-all	Alpha gewinnt still (CONVERGENCE)	STABLE_AMBIGUITY + UNRESOLVED_PENDING_RULE, winning_id=None
Branch	implizites (ca, cb)-Tuple	BranchNode First-Class mit branch_id, merge_policy, lifecycle
D.5 Seal	Φ ≥ 0.65 hard blocking	ADVISORY ONLY — nicht-blockierend
branch_triggers	list[tuple[Claim,Claim]]	list[BranchNode]

Sprint 2

Komponente	Alt (v2.1)	Neu (v2.2)
schema.py	Claim/Patch implizit	EpistemicIdentity-Doktrin als normative Klasse
relations.py	(neu)	RelationsMatrix: maschinen-checkbare Zulässigkeitsmatrix
audit.py	1 Klasse, 5 Blocks gemischt	ThreeLevelAudit: Patch / Claim / Graph getrennt
Uncertainty	modality-basierter Trigger	EpistemicIdentity.uncertainty_required():

		präzise 3-Bedingung-Logik

Sprint 3

Komponente	Alt (v2.1)	Neu (v2.2)
builder.py	Kein Konfidenzgrad	MappingConfidence, ConceptMappingResult
sources.py	map_term() ohne Konfidenz	map_term() gibt ConceptMappingResult zurück
diff.py	Keine Bias-Metadaten	adjudication_rule, winning_builder, bias_tag
schema.py	validate() ohne Uncertainty-Regel	EpistemicIdentity.uncertainty_required() statt Modality-Heuristik
audit.py	Block V Heuristik	EpistemicIdentity-Regel, advisory für optionale uncertainty

Ende von Annex H — v2.2 Structural Changes (Sprint 1–3)